SnapPark

Abstract

The continuing growth of urban traffic has made illegal parking a persistent civic problem: vehicles left on pedestrian crossings, pavements and no-stopping zones endanger vulnerable pedestrians, obstruct emergency access and erode the quality of public space. Enforcement still depends largely on the physical presence of traffic wardens, whose reach is bounded by numbers and hours of duty. This dissertation presents SnapPark, a cloud-based platform that lets citizens report suspected parking violations by submitting one or more photographs through a web application, after which the submission is examined automatically and a structured, reasoned opinion on whether a violation is present is produced and stored. SnapPark is built as a set of independently deployable services coordinated behind a single entry point, and is engineered for scalability, fault isolation, security, auditability, extensibility and maintainability. The report situates each architectural decision against the alternatives that were considered and rejected, documents the incremental development of the system across four increments, and evaluates the finished platform against the objectives established at the outset. The work shows that combining a service-based architecture with a modern AI back-end is a viable and extensible foundation for a socially useful civic-technology system.

Chapter 1 — Introduction

In recent years the number of vehicles on urban roads has grown sharply, and with it the frequency of illegal parking. A car left on a pedestrian crossing, a pavement or a no-stopping zone is not a trivial inconvenience: it forces pedestrians into the road, creates direct danger for people with reduced mobility, parents with prams and visually impaired citizens, delays emergency vehicles, and contributes to a steady decline in the quality of urban life. The conventional way of dealing with this is enforcement by traffic wardens who patrol on foot or by vehicle. Their effectiveness is intrinsically limited by how many of them there are, where they happen to be and when they are on duty. At night, in outlying neighbourhoods, or during peak periods, a large proportion of violations are simply never recorded.

At the same time, several technological developments have matured to the point where they can be combined into a practical response. Smartphones with capable cameras are now everywhere, and citizens are increasingly willing to report problems in their neighbourhood when they are given a convenient way to do so. Commercial vision-capable AI models can now interpret a photograph and produce a written justification for their conclusion without a dedicated team having to collect data and train a bespoke model. And the software engineering community has converged on a service-based architectural style that allows distributed systems to scale and evolve in parts rather than as a single block.

SnapPark brings these together. A citizen who sees a suspected violation opens the SnapPark web application on their phone or computer, takes or selects a photograph, optionally adds the licence plate, and submits it. Behind the web client sits a set of cooperating back-end services: a single gateway verifies and forwards every request, an authentication component confirms the user's identity, an analysis component checks the image and obtains a reasoned verdict from a vision AI model, and a notification component informs the relevant parties once the case has been processed. The structured result — whether a violation was confirmed, the type, a confidence value and a short explanation — is stored so that it can be inspected later.

It is worth stating clearly what the project is and is not. SnapPark is not an attempt to build a production-grade legal replacement for a human warden, nor does it claim that an AI verdict is sufficient on its own to issue a penalty. It is a demonstration that a well-engineered, service-based architecture combined with a modern AI back-end is a sound and extensible foundation on which a civic-reporting platform of this kind can be built, with a human retaining the decision to escalate a case to the authorities.

1.1 Project Aim

The aim of SnapPark is to design, implement and evaluate a cloud-based, service-oriented platform that allows citizens to report suspected illegal parking by submitting one or more images through a web application, and that uses an AI vision model to produce a structured,

reasoned and human-readable opinion on whether a violation has occurred. The platform should demonstrate that it can scale unevenly across its parts, isolate failures, secure every request, keep an auditable record of what happened, accept new capabilities without disturbing what already exists, and remain small enough in each part to be understood and maintained by one developer. It should be runnable on a single machine for development and deployable to a cluster for production.

1.2 Objectives

1. **Scalability.** The system must be able to grow unevenly: the parts that experience heavier load — for example the analysis pipeline during a surge of reports — must be scalable on their own, without redeploying the whole platform.
2. **High availability.** The failure of any one component must not propagate. A failed notification component must not stop citizens submitting reports; a failed authentication component must not take down analysis for users whose sessions are already valid.
3. **Security.** Every request must be authenticated before it reaches any business logic, credentials must never be stored in plain text, and access tokens must be short-lived and revocable.
4. **Auditability.** Every significant change of state — a case being created, analysed, escalated or cancelled — must leave a permanent trace that can be inspected afterwards for legal, operational and debugging purposes.
5. **Extensibility.** Adding a new notification channel, a new event consumer, or even a new service must not require changing the existing services.
6. **Maintainability.** Each service must be small enough to be understood in full by one person, have a clearly defined responsibility, and be independently testable and deployable.

1.3 Report Structure

Chapter 2 reviews the literature and the technologies behind SnapPark, comparing each major decision against the alternatives that were rejected. Chapter 3 gives a description of the system together with its functional and non-functional requirements. Chapter 4 explains the development process that was followed and the way risk was managed. Chapters 5 to 8 each document one increment — its analysis, design, implementation, testing and evaluation — covering, respectively, identity and the gateway; violation analysis and the case lifecycle; event-driven notifications and the orchestrated saga; and the web client, semantic search and cloud deployment. Chapter 9 evaluates the finished system against the objectives above. Chapter 10 reflects on the challenges, the lessons learned, and the future work that could extend the platform.

Chapter 2 — Literature Review

This chapter reviews and compares the technologies, architectural patterns and design patterns behind SnapPark. It is organised around the major decisions made during the project: the overall architecture (§2.1), the API Gateway (§2.2), managing data across independently owned services (§2.3), communication between services (§2.4), packaging and deployment (§2.5), analysis of uploaded images (§2.6), retrieval of semantically similar past cases (§2.7), and the implementation technologies (§2.8). Each section sets out the options that were considered, weighs their strengths and weaknesses, and ends by justifying the option chosen for SnapPark.

2.1 Architectural Paradigms

This section traces the trajectory from monolithic systems, through Service-Oriented Architecture, to the Microservices architecture finally adopted, and the technical pressures that drove each transition.

2.1.1 Monolithic Architecture

A monolithic architecture treats the whole system as one cohesive unit, developed and deployed as a single artefact [1]. The user interface, business logic, data access layer and infrastructure code all share a process and address space and typically a single database, with components communicating through in-process calls. This is a straightforward model and performs well because there is no network between components.

Its appeal is operational simplicity: one process to deploy, one log stream, one set of credentials, one artefact to test end to end [1]. For small systems or first versions these are real advantages. The weaknesses appear as the application grows. A minor change forces a rebuild, retest and redeploy of the entire application, slowing releases. Components are tightly coupled at code and database level, so one area cannot easily change without affecting another [2]. Scaling is coarse-grained — the whole application must be scaled when only one part is under load — giving a poor price-to-performance ratio. A defect such as a memory leak in one component can crash the whole process. And the entire application is bound to one language and framework, making migration disruptive and expensive [15].

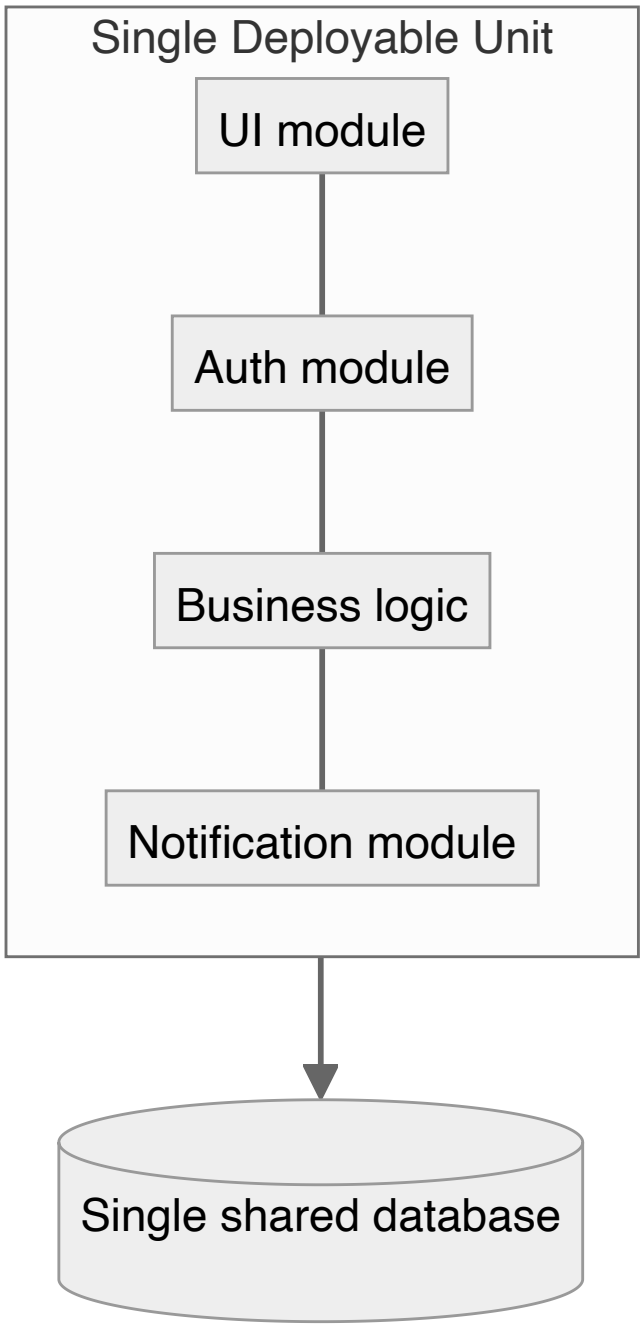

***Figure 2.1** — Monolithic architecture — a single deployable unit containing every module, sharing one database.*

2.1.2 Service-Oriented Architecture (SOA)

SOA emerged to address these weaknesses by decomposing a large application into coarse-grained, reusable services that communicate over a network, historically through XML-based SOAP messages routed by an Enterprise Service Bus (ESB) [4, 5]. The ESB handles routing, message transformation, security-policy enforcement and a single point of audit.

SOA's benefits centre on reusability and central governance: once a service is published, many consumers can use it, and the ESB applies a uniform policy to every published service. But the ESB is a single point of failure and a performance bottleneck, since all traffic passes through it [6]. The coarse-grained nature of SOA services limits reusability in practice and reintroduces the very coupling SOA set out to remove. The WS-* specifications and the verbose SOAP/XML stack also impose heavy computational overhead compared with the JSON-over-HTTP that most modern web development has adopted [5, 6].

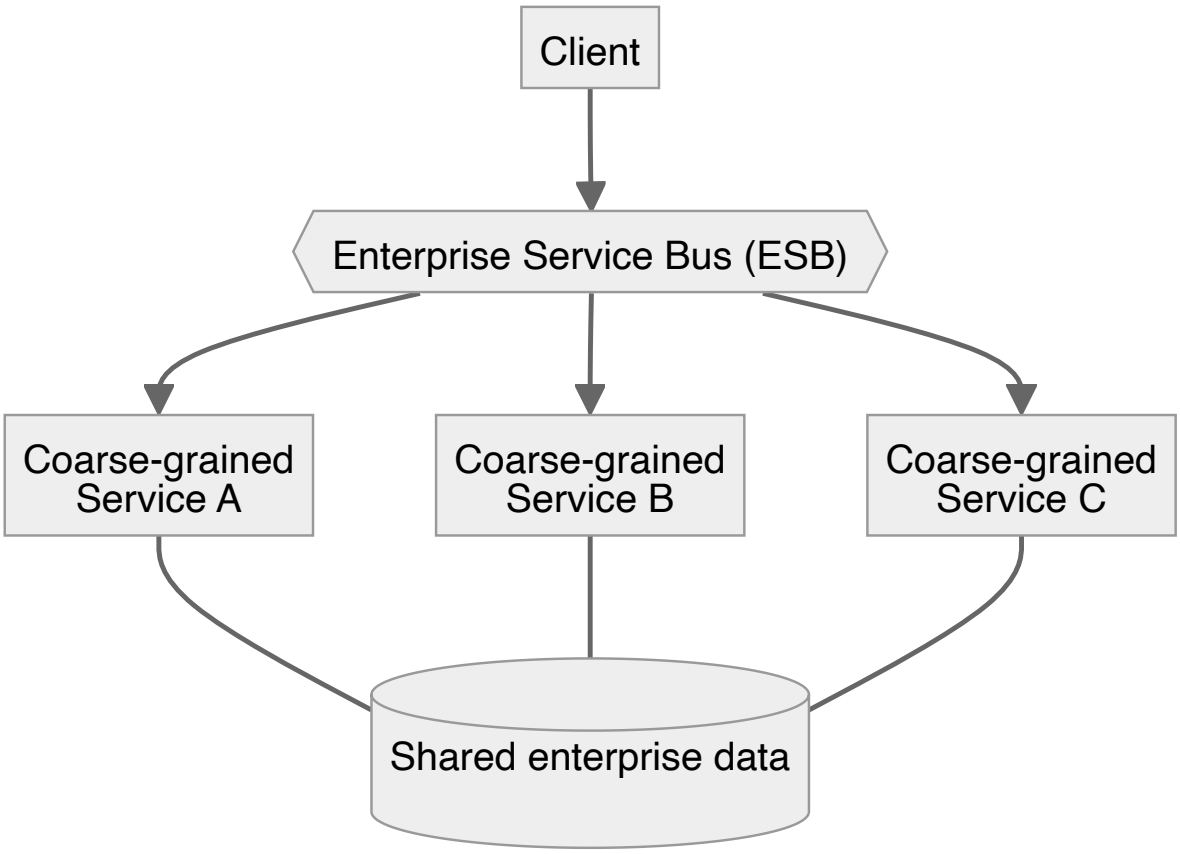

***Figure 2.2** — Service-Oriented Architecture — coarse-grained services communicating through a centralised ESB.*

2.1.3 Microservices Architecture

The microservices style decomposes an application into many small, independent services, each performing a single well-defined business function [15]. Each service owns its data, exposes a network-accessible interface (typically JSON-over-HTTP or an asynchronous message contract), and can be deployed, scaled and updated on its own. Unlike SOA's centralised bus, microservices communicate directly and through lightweight brokers, pushing intelligence to the endpoints rather than concentrating it in middleware — the shift often described as moving from "smart pipes, dumb endpoints" to "dumb pipes, smart endpoints" [15].

Independent deployability brings fine-grained scaling, technological diversity and isolated fault domains: each service can be released on its own schedule, scaled on its own demand curve, written in the most suitable language, and insulated from the failures of others [2, 16]. The cost is operational complexity — automated deployment, distributed observability, request tracing and the management of distributed communication — none of which a monolith requires [2]. Distributed data management is also harder, and introducing a network creates new failure modes such as partial failure, retries and eventual consistency [5].

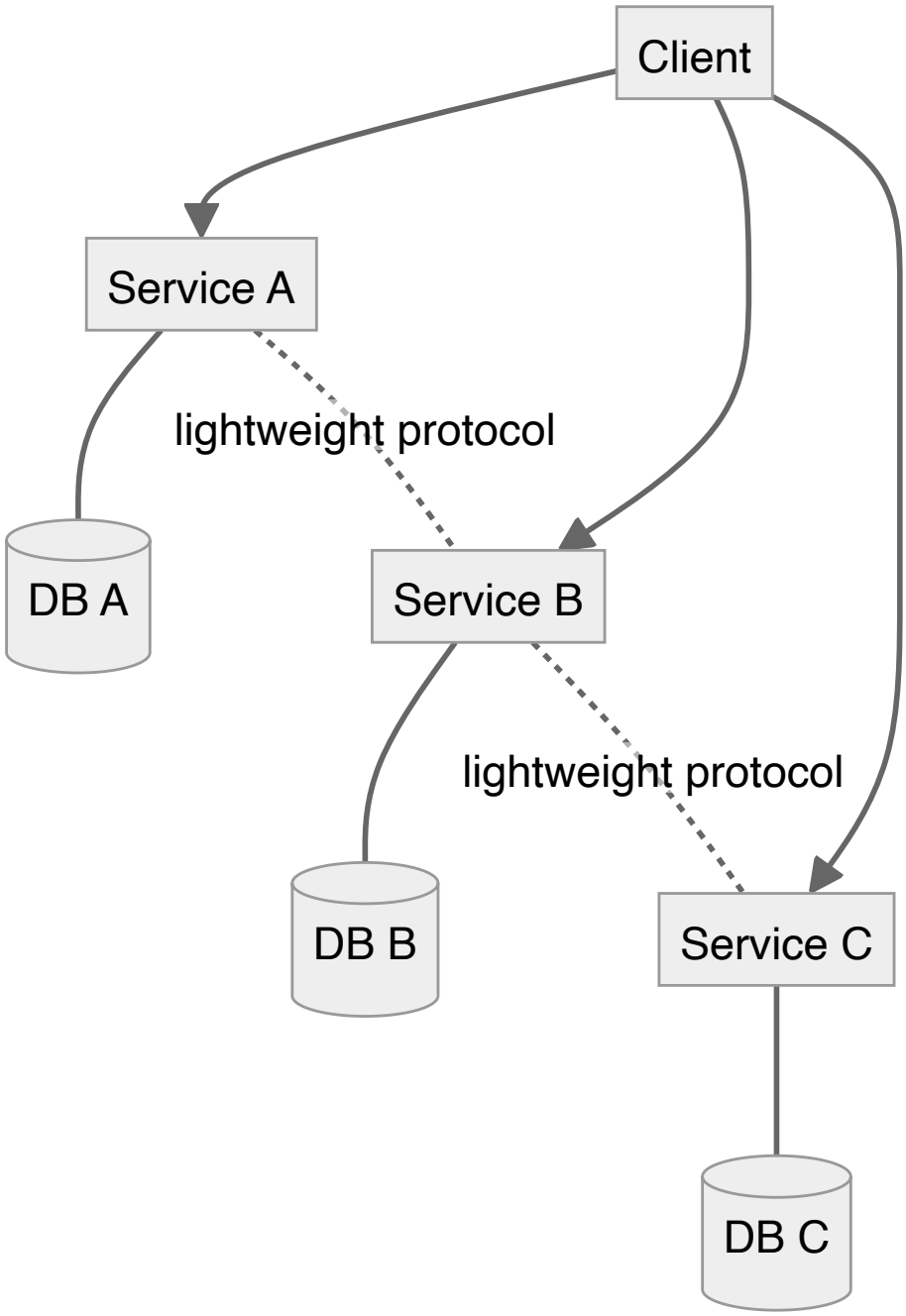

***Figure 2.3** — Microservices architecture — small independent services, each owning its database, communicating over lightweight protocols.*

2.1.4 Architectural Decision

SnapPark is built on a microservices architecture. A monolith was considered but rejected because the workload is fundamentally asymmetrical: the analysis component is dominated by computationally intensive image processing and externally provided AI inference, while authentication and notification are lightweight and low-latency, so deploying and scaling all of them as a single artefact would be wasteful. SOA was rejected because a centralised bus would reconcentrate the coupling the project wants to avoid, and because the verbose XML/SOAP stack is a poor fit for an interactive web application that talks to browser-based clients in JSON.

The decision is expressed in the source code as three independent services — Authentication, Violation Analysis and Notification — each with its own `package.json`, Dockerfile, source tree and PostgreSQL instance (`postgres_auth`, `postgres_case`, `postgres_notifications`). Services are coordinated locally through `deployment/docker-compose.yml` and in production through the manifests in `deployment/kubernetes/`. The operational complexity of microservices is mitigated by the orchestration and automation discussed in §2.5.

2.2 API Gateway

An API Gateway is the server-side intermediary between external clients and the internal services of a distributed system. It is the single entry point for all external traffic: every microservice is reachable through one address, and the gateway processes each request, enforces cross-cutting policies, and routes it to the appropriate downstream service. Typically it handles routing, authentication and authorisation, rate limiting, request and response transformation, request aggregation and TLS termination.

Centralising these functions relieves each microservice from implementing them separately and gives a consistent treatment of every request. The gateway also decouples the addresses and interfaces that clients use from the internal layout of the services: clients are unaffected when services are split, merged, renamed or relocated, as long as the gateway configuration is updated. The gateway is often contrasted with an ESB; the difference is one of intelligence. An ESB carries business logic for transformation and orchestration and tends to grow into a large, complex component that is hard to change, whereas the API Gateway is deliberately lightweight.

SnapPark's Express API Gateway [42] runs on port 3000 and is the only way for clients to reach the backend. A custom gateway in Express was chosen over an off-the-shelf product because the requirements were modest — validate a JWT against the authentication service and route to three backends — and a heavyweight commercial or self-hosted gateway would have added operational complexity, an external dependency or a licence cost, none of which is reasonable for a student project. The implementation in `services/api-`

`gateway/src/index.js` uses an `authenticate` middleware on all protected routes, Helmet [45] for security headers, and `express-rate-limit` [33] for per-IP throttling, before routing requests to the three configured backends.

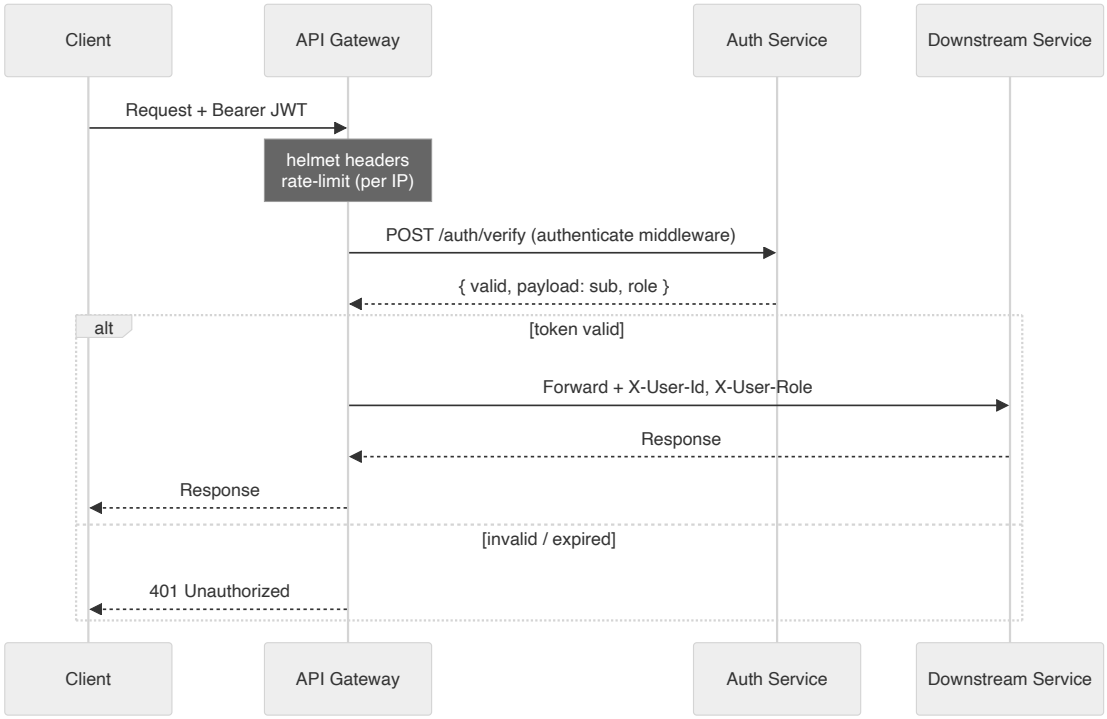

***Figure 2.4** — Request flow through the API Gateway — client → gateway (helmet, rate-limit, JWT validation) → downstream service.*

2.3 Data Management in Microservices

Data management is one of the hardest aspects of building a microservices system [10, 16, 48]. In a monolith, a single shared database and its transaction machinery keep data consistent. In microservices, a shared database is an anti-pattern: it tightly couples services so that a change in one can force changes in others, and a slow or heavy query in one service can degrade all the others.

The Database-per-Service pattern gives each service its own database, accessible to other services only through the owning service's API [13]. This prevents coupling at the data layer, mirrors the encapsulation that microservices provide at the code level, and lets each service choose the most suitable database technology, evolve its schema independently, and be isolated from the performance or failure of any other database. The price is that an operation spanning several services can no longer be a single unit of work, since the databases are no longer coordinated. The system must therefore choose between a distributed-transaction protocol such as two-phase commit — costly and failure-prone — and eventual consistency [48]. The Saga pattern is the usual way of handling multi-step processes under eventual consistency [10, 26].

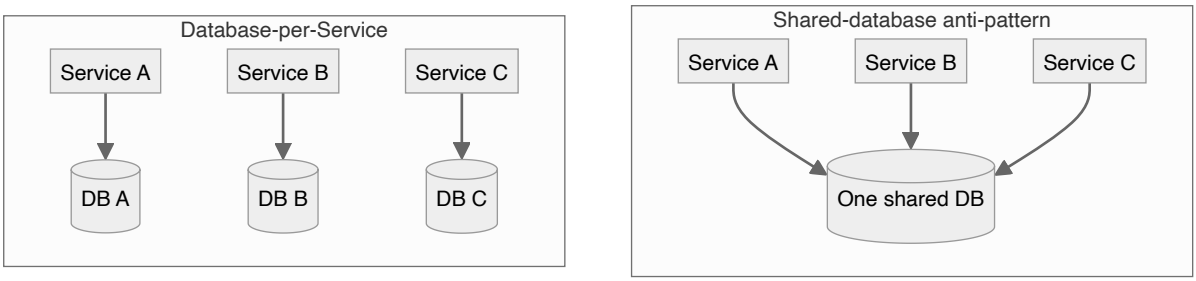

***Figure 2.5** — Shared-database anti-pattern vs. Database-per-Service — each service owning a private store.*

2.3.1 The Saga Pattern

The Saga was defined by Garcia-Molina and Salem in 1987 [26] as a sequence of local transactions, each executed by a single service, where the success of one step triggers the next and, if any step fails, compensating actions reverse the previous steps in reverse order. A saga thus lets an operation spanning several services behave, from the user's point of view, as a single unit of work without any distributed-transaction infrastructure [10, 26].

There are two ways to implement a saga [10]. In **choreography**, each service publishes an event when it finishes its step; the next service subscribes to that event and performs its own step. There is no central conductor — the saga is simply the sum of the published events and their subscriptions. In **orchestration**, a single coordinator drives each service to perform its step and records whether it succeeded; on failure, the coordinator performs the compensations. Choreography reduces coupling and suits simple linear sagas, but there is no single place that defines how the whole saga behaves: tracing it means following subscription events across several codebases [10]. Orchestration increases coupling because of the coordinating code, but it makes the saga a first-class object that can be defined and viewed as a whole — the step definitions, the persisted saga state and the compensations all live in one place [10].

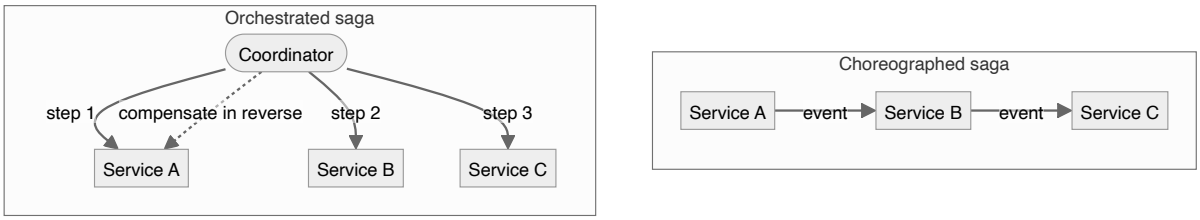

***Figure 2.6** — Choreographed saga (events between services) vs. orchestrated saga (a central coordinator driving steps and compensations).*

2.3.2 Choice

SnapPark adopts the Database-per-Service pattern [13], which gives the greatest benefit for independent deployment. The Shared-Database pattern was judged too tightly coupling and would have undermined independent deployability, a core goal [10, 13]. SnapPark runs three services, each on its own PostgreSQL database: `postgres_auth` holds `snappark_auth`,

`postgres_case` holds `snappark_case`, and `postgres_notifications` holds `snappark_notifications`. A service reaches another service's data only through the owning service, each with its own credentials.

Case creation requires image analysis, case persistence, image persistence, embedding generation, audit logging and notification dispatch. This workflow is implemented as an **orchestrated saga** [10, 26], because it consists of six interdependent steps whose compensations must run in reverse on failure, and orchestration lets the whole workflow be documented as a single formal artefact (`caseCreationSaga.js`) with its compensations alongside it. After each step, the saga's current state is persisted to the `sagas` table so it can be inspected, resumed or compensated after a failure [10, 34].

2.4 Communication Between Services

The way services communicate affects coupling, latency, fault tolerance and operational complexity [10, 48]. Communication is either synchronous — the caller blocks until it receives a response — or asynchronous — the sender posts a message and continues without waiting [48].

2.4.1 Synchronous Communication

Synchronous communication is simple: one service calls another over the network and blocks until it receives the result. The two common protocols are HTTP-based REST with JSON, and gRPC [47], which uses HTTP/2 with Protocol Buffers. REST is the de facto standard for public web APIs and is supported natively everywhere; gRPC offers higher performance and stronger typing but needs more tooling and produces non-human-readable wire traffic [47]. The advantage of synchronous calls is conceptual simplicity and an immediate response; the disadvantage is that the caller is tightly coupled to the latency and availability of the callee, so one slow service can stall the whole system [34, 48].

2.4.2 Asynchronous Communication

Asynchronous communication decouples sender from receiver in time [48]. The sender posts a message to a broker and continues; the receiver consumes and processes it whenever it is ready, possibly much later. The broker must store the message durably, route it to the right consumers, and ensure it is not lost [23, 40]. The two most widely used brokers are RabbitMQ (an AMQP implementation) [23] and Apache Kafka (a distributed log) [48]: RabbitMQ emphasises rich routing, per-message acknowledgement and operational maturity [23, 40], while Kafka emphasises throughput, retention and replay, which suits very high scale where consumers re-read historical events [48].

RabbitMQ offers several exchange types [23]: a **direct** exchange routes by exact routing-key match; a **fanout** exchange ignores the key and broadcasts to every bound queue; a **topic** exchange routes by pattern using the wildcards `*` (one word) and `#` (zero or more words),

letting consumers subscribe to broad or narrow classes of events with one declarative pattern; and a **headers** exchange routes on header attributes and is rarely used in practice. The benefits of asynchronous communication are loose coupling, fault tolerance and natural one-to-many delivery [10, 48]; the costs are added operational complexity and the cognitive difficulty of reasoning about separated cause and effect [34, 48].

2.4.3 Conclusion

SnapPark uses RabbitMQ with a topic exchange as its primary inter-service communication method. RabbitMQ was chosen over Kafka because of its lower operational demands — it does not need Kafka's log-replay capability — and because its rich routing fits the various events published across the case lifecycle [23]. A topic exchange was chosen because it supports both broad and narrow subscriptions from one declarative pattern [48]: a direct exchange would require binding each routing key separately; a fanout exchange would deliver every message to every subscriber regardless of interest; and a headers exchange would add indirection with no corresponding benefit. The topic exchange also serves the extensibility objective: new services can subscribe to a broad or narrow class of events without any change to the publisher.

The code reflects this exactly. In `services/violation-analysis-service/src/rabbitmq.js` the exchange is declared as `'topic'`, and the saga publishes `case.created`, `case.reported`, `case.resolved` and `case.cancelled`. The Notification Service binds queues to these routing keys and publishes `notification.failed` back to the exchange when every delivery channel for a case fails, closing the saga's compensation loop. Synchronous HTTP is used in only one place: the gateway calls the Authentication Service's `/auth/verify` endpoint to validate a JWT, because that result is needed immediately before a request can proceed.

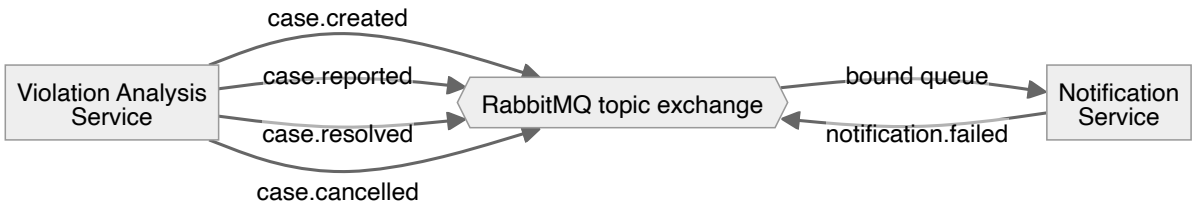

***Figure 2.7** — SnapPark event flow — the saga publishing `case.*` events to the topic exchange, the Notification Service consuming them and publishing notification.failed back.*

2.5 Packaging and Deployment

A microservices system is many independently deployable processes [10, 15], typically packaged with containerisation and managed with container orchestration.

2.5.1 Containerisation with Docker

A container is a minimal, self-contained execution unit bundling an application with its libraries and dependencies. Containers share the host kernel, so no runtime libraries are duplicated across containers on the same host [8, 49], and they can be created and destroyed in seconds, unlike virtual machines [8]. Docker [8, 49] is the leading container technology: an image is built from a Dockerfile that describes it layer by layer, and once built it runs on any Docker host. Because the application is bundled with its dependencies, the development and test environment closely matches production [8, 49].

2.5.2 Container Orchestration with Kubernetes

A production environment of many containers cannot be managed by hand. Container orchestration automates the provisioning, scheduling, scaling and management of containers across a cluster, including restarting failed containers, scaling with traffic, rolling out new versions without downtime, and managing networking and storage [14, 35]. Kubernetes is the industry-standard platform [14, 35]: deployments are described declaratively in YAML manifests stating the desired state, and Kubernetes continually reconciles the actual state with it [14], while providing service discovery, load balancing, secret and configuration management, horizontal autoscaling and rolling updates [35]. Docker Swarm (simpler but a much smaller ecosystem) and HashiCorp Nomad (lighter but a smaller community) were considered; Kubernetes was chosen as the leading platform with the highest-quality operational tooling [35].

2.5.3 Choice

SnapPark uses Docker [8] to containerise each of the three microservices and the API Gateway; every component has its own Dockerfile and can be built and run independently. For local development the components are wired together with Docker Compose (`deployment/docker-compose.yml`). For production the project includes a full set of Kubernetes manifests [14, 35] in `deployment/kubernetes/` — deployments, services, ingress, config maps, secrets, horizontal pod autoscalers, and StatefulSets for the databases and broker. This follows the standard cloud-native split of Docker Compose for development and Kubernetes for production [25, 35]: Compose is quick locally, but only Kubernetes answers the production questions of progressive rollout, autoscaling and health-based replacement, at the cost of greater complexity as a local tool [14].

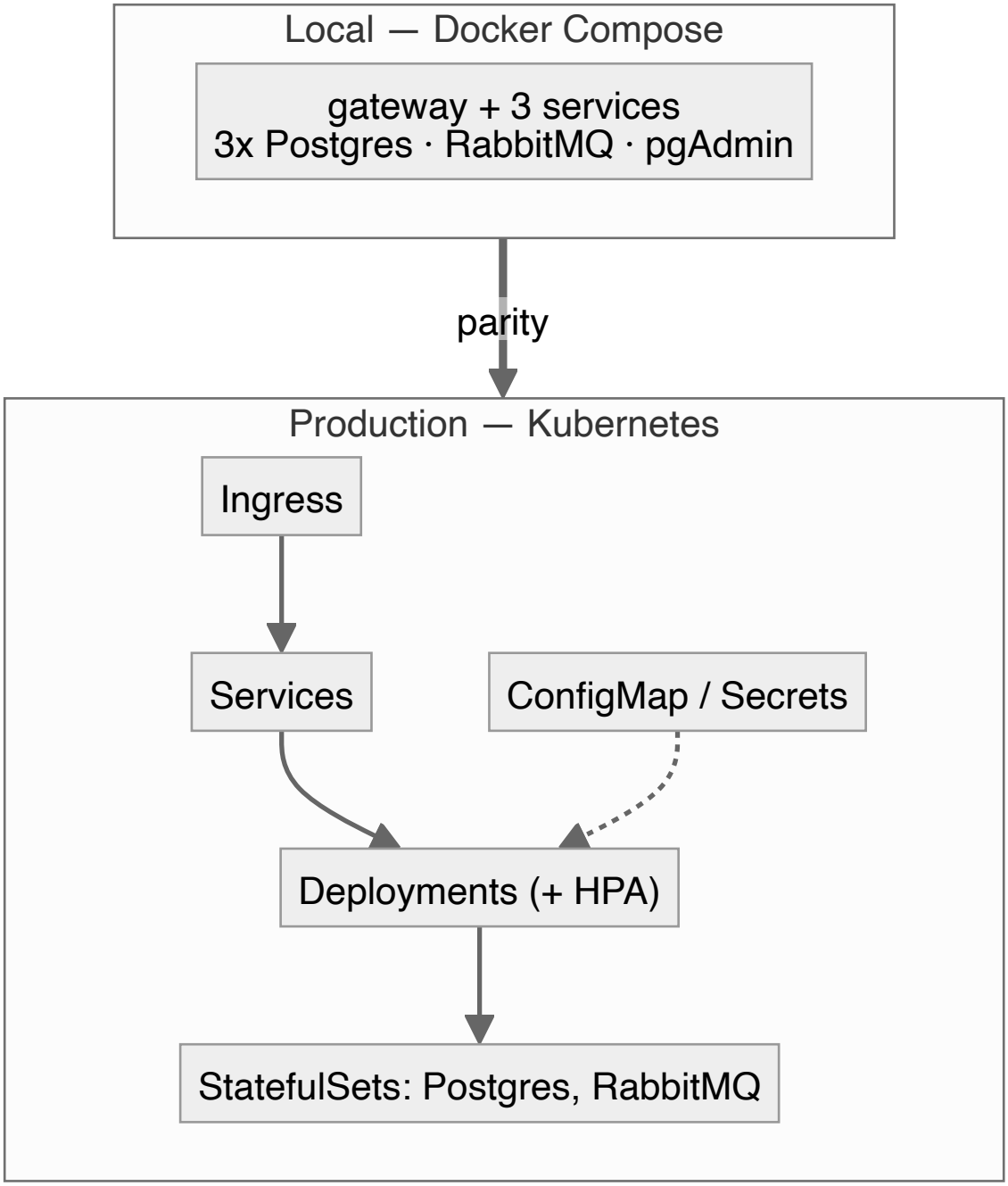

***Figure 2.8** — Deployment topology — Docker Compose locally vs. the Kubernetes cluster in production.*

2.6 AI-Aided Image Evaluation

When a citizen submits a photograph, SnapPark's analysis component must decide whether the vehicle is illegally parked, classify the violation if there is one, assign a confidence score, and produce a short natural-language justification. Two families of technology were considered: traditional machine-learning computer vision, and vision-enabled foundation models.

2.6.1 Traditional Computer Vision

The traditional approach trains a bespoke convolutional neural network on a labelled image dataset, an approach with a long history in transport computer vision such as licence-plate recognition [55]. It offers maximum control over the model's behaviour, predictable inference time, and no dependence on an external provider. But it is costly: it requires a large, well-curated, balanced dataset of the target violations in the relevant jurisdiction, a machine-learning team to design and train the model, and dedicated GPU infrastructure — and once built, the classifier only assigns images to the fixed categories it was trained on. Producing a natural-language explanation rather than just a label is a separate research problem [53]. This approach was rejected because none of its prerequisites realistically existed in an undergraduate project; the project's contribution lies in the architectural integration of generative AI into a civic platform, not in building a new vision model.

2.6.2 Vision-Enabled Foundation Models

Since around 2023 a class of multimodal foundation models can perform sophisticated image tasks with no custom training [30, 53]. The principal options were: **Google Gemini** (the `gemini-2.5-flash` and `gemini-1.5-flash` families) [24, 30] — fast, inexpensive, with native multimodal input and strict-schema structured JSON output, and from the same vendor as the `text-embedding-004` model used in §2.7; **OpenAI GPT-4V** (the GPT-4o family) [51] — strong image reasoning and mature tooling, but a higher per-call cost and a separate vendor account; and **Anthropic Claude** (3.5 Sonnet and above) [52] — strong reasoning and the most rigorous safety tuning, but slower in the lightweight tier and, at the time of the decision, unable to guarantee the same structured output as Gemini. The benefits of this approach are no training data, no machine-learning expertise and no custom infrastructure [30, 53]; the model returns a structured decision and an explanation in a single call. The drawbacks are dependence on a third party, per-call cost, network latency, and the tendency of large models to be confidently wrong at times — the last mitigated by treating the verdict as an opinion, showing the justification to the user, and requiring a human admin to approve escalation to an authority.

2.6.3 Selection of Model

SnapPark uses Google Gemini, specifically `gemini-2.5-flash`, through the `@google/generative-ai` SDK [24], configured with the `GEMINI_API_KEY` environment variable. Three reasons drove the choice: its native multimodal input and structured-JSON output let the analysis be completed in a single API call; it reasons about general-purpose visual criteria far better than a model a single student could train [30]; and its price and latency suit an interactive web application. GPT-4V [51] and Claude [52] were passed over on cost and structured-output fit, not capability — either could have met the criteria. In `services/violation-analysis-service/src/gemini.js`, each image is supplied as Base64

`inlineData` with a prompt specifying the JSON schema, and the response is parsed into `{violationConfirmed, violationType, confidence, explanation}`, which feeds the next saga step.

```
const SINGLE_IMAGE_PROMPT = `You are an expert traffic warden and parking enforcement
        officer.
Analyse the provided image and determine whether an illegal parking violation is
        occurring.

Respond ONLY with a single valid JSON object — no markdown, no extra text — in exactly
        this shape:
{
  "violationConfirmed": <true | false>,
  "violationType": "<short description of violation, or null if none>",
  "confidence": <float 0.0–1.0>,
  "explanation": "<one or two sentences explaining your determination>"
}

Guidelines:
- Set violationConfirmed to true only when you can clearly see a parking violation.
- violationType examples: "blocking fire hydrant", "double parking", "no stopping
        zone",
  "expired meter", "bus stop obstruction", "pavement parking", "no parking zone".
- confidence should reflect how certain you are (e.g. 0.95 = very certain, 0.5 =
        uncertain).
- If the image does not show a vehicle or road scene, set violationConfirmed to false
        and
  explain what you see instead.`;

// ……

const parseResponse = (text) => {
  const clean = text.replace(/^```(?:json)?\s*/i, '').replace(/\s*```$/, '').trim();

  let parsed;
  try {
    parsed = JSON.parse(clean);
  } catch {
    throw new Error(`Gemini returned non-JSON response: ${text.slice(0, 200)}`);
  }

  return {
    violationConfirmed: Boolean(parsed.violationConfirmed),
    violationType:      parsed.violationType ?? null,
    confidence:         Number(parsed.confidence ?? 0),
    explanation:        String(parsed.explanation ?? ''),
```

```
  };
};
```

***Listing 2.1** — Prompt and JSON-schema parsing in gemini.js.*

2.7 Semantic Similarity and Vector Databases

A useful function for a case-management system is to take a new case and find previous cases that are semantically similar. This section reviews the technologies for similarity search.

2.7.1 Text Embeddings

A text embedding is a fixed-length numerical vector representing the meaning of a piece of text in a high-dimensional space [48]. A good embedding places semantically similar texts closer together than unrelated ones. Embeddings are produced by purpose-built models such as Google's `text-embedding-004` (768 dimensions) [24], OpenAI's `text-embedding-3-small` (1536 dimensions) and Cohere's `embed-english-v3.0` (1024 dimensions). Because semantic similarity becomes geometric proximity, the texts most similar in meaning to a query are simply those whose vectors are nearest — so a search for "blocked sidewalk" can return cases about obstructed pedestrian paths even with no shared keywords.

2.7.2 Vector Databases

A vector database stores and efficiently searches high-dimensional vectors, and comes in two forms. **Dedicated vector databases** such as Pinecone [74], Weaviate [75], Milvus [76] and Qdrant [77] are purpose-built and give the best performance at the largest scales, but add a separate piece of infrastructure to provision, secure, maintain and keep consistent with the other data stores. **Existing-database extensions** such as pgvector for PostgreSQL [61] add vector storage, indexing and search to a general-purpose database alongside the related relational data, with no new datastore. Both use approximate-nearest-neighbour indexes — Hierarchical Navigable Small World graphs (HNSW) [73] or inverted-file indexes — to reduce search from linear time to roughly logarithmic; HNSW tends to give lower query latency and higher recall at the cost of greater memory use [73].

2.7.3 Distance Metrics

Three distance metrics are common. **Cosine distance** measures the angle between two vectors, ignoring magnitude; since the magnitude of embedding vectors carries little semantic meaning and their direction carries it, cosine is almost universally recommended for text embeddings [61]. **Euclidean distance** measures straight-line distance and is affected by both magnitude and direction. **Inner-product distance** uses the dot product and suits cases where both direction and magnitude are meaningful.

2.7.4 Choice

SnapPark uses PostgreSQL with the pgvector extension [44, 61], an HNSW index [73], cosine distance [24], and Google's `text-embedding-004` model (768 dimensions) for embedding case verdicts. pgvector was chosen over the dedicated databases [74–77] because the Violation Analysis service already stores its relational data in PostgreSQL [44]: the verdict embedding lives in the same `cases` row as the rest of the case, a single SQL query retrieves a case and its nearest neighbours, transactional consistency is preserved, and no new infrastructure is introduced. The code supports this at `db.js:39` (enabling the extension), `db.js:180–192` (adding the 768-dimension embedding column and the HNSW index `idx_cases_embedding_hnsw ON cases USING hnsw (embedding vector_cosine_ops)`), and `db.js:328–360` (the `<=>` cosine-distance similarity query). HNSW was chosen because the dataset is small enough that its memory cost is not an issue and it gives the lowest latency for the required recall. Cosine distance was chosen because `text-embedding-004` is intended for it and the vector magnitude carries no useful meaning here [61]. `text-embedding-004` was chosen over OpenAI, Cohere and open-source alternatives because the project already uses Google for image analysis (§2.6), so one provider means fewer dependencies, simpler billing and credential management, and no self-hosted embedding model to maintain.

2.8 Implementation Technologies

2.8.1 Server-Side Runtime: Node.js

Node.js [56] is a JavaScript runtime on the V8 engine with an event-driven, non-blocking I/O model well suited to network-bound services. Python with FastAPI [57] was considered (excellent ecosystem, but the GIL and ASGI model make high-concurrency network services awkward), as were Go [59] (good performance, smaller ecosystem, steeper learning curve) and Java with Spring Boot [58] (mature but with a larger memory footprint and startup time than the size of SnapPark's services justifies). Node.js was chosen because it lets the whole stack — the three services, the gateway and the Next.js front-end — be written in one language, reducing context switching, allowing shared utilities and simplifying onboarding.

2.8.2 HTTP Framework: Express

Express [42] was chosen over Fastify [60] and NestJS. Fastify offers higher throughput but a smaller ecosystem and a less familiar API; NestJS is architecturally opinionated but adds complexity not warranted at SnapPark's scale. Express has a long history and a mature middleware ecosystem, including the packages SnapPark relies on: Helmet for security headers [45], express-rate-limit for per-IP throttling [33] and multer for multipart form data [66].

2.8.3 Front-End Framework: Next.js

Next.js 14 (App Router) [43] is a React framework supporting both server- and client-rendered content from one codebase. React with Vite [70] was considered (would require building routing, build and server-rendering tooling from scratch), as were Vue with Nuxt (comparable but less familiar) and SvelteKit (promising but, at the time, a smaller ecosystem). Next.js was chosen for its opinionated structure that gets a working application running with little setup, the size of the React ecosystem, and server rendering for fast initial loads. Styling and the responsive breakpoints for the upload and case-view pages use Tailwind CSS.

2.8.4 Database: PostgreSQL

PostgreSQL [44] is a mature, open-source object-relational database with full SQL support, strong transactional guarantees, a rich type system and an extensive extension ecosystem including pgvector (§2.7) [61]. MySQL/MariaDB (a smaller extension ecosystem and no pgvector equivalent), MongoDB (schema-flexible but without strong transactions or SQL) and SQLite (a fine embedded database but poorly suited to a multi-process, multi-host architecture) were considered [48]. PostgreSQL was chosen for its relational rigour, native JSON support, vector search via pgvector, and proven operational reliability [44, 48].

2.8.5 Authentication: JWT and bcrypt

Authentication uses JSON Web Tokens (JWTs) rather than server-side sessions. A session stores state on the server, requiring sticky sessions or a shared session store; a JWT carries the user's identity and a signed assertion within the token, making it stateless, so services can validate it without a central store [31, 63]. This suits microservices: the gateway validates the JWT once per request and forwards only the user's identity to downstream services in the `X-User-Id` and `X-User-Role` headers. Refresh tokens limit each access token's lifetime and allow revocation through rotation [22, 31].

Passwords are stored hashed with bcrypt [32], a deliberately slow adaptive hash with a configurable work factor that resists brute-force attacks. (Argon2 [64, 65], the Password Hashing Competition winner, was a stronger theoretical alternative, but bcrypt via the well-supported `bcryptjs` library was chosen for its maturity and zero native-build dependencies, which keeps the service easy to containerise.)

2.8.6 Testing

Back-end services are tested with Jest [67] and Supertest [68] for HTTP-level assertions in the authentication service, and with Vitest [69] and Supertest in the violation-analysis and notification services (Vitest integrating with the Vite tooling Next.js uses internally); the front-end is tested with Vitest [69] and React Testing Library. Coverage is measured with Istanbul [71], reporting the four IEEE-982 [39] metrics used in Chapter 9.

2.8.7 Summary

SnapPark uses a Node.js [56] back-end with Express [42], and a Next.js [43] front-end with Tailwind CSS. Each service has its own PostgreSQL [44] database, with pgvector [61] enabled for the Violation Analysis service. JWTs [31, 63] secure authentication and passwords are hashed with bcrypt [32]. The whole system is containerised with Docker [8], orchestrated locally with Docker Compose and in production with Kubernetes [14, 35]. The stack is the product of weighing each technology against the alternatives, balancing developer productivity, operational simplicity and engineering quality.

Chapter 3 — System Description and Requirements

3.1 System Overview

SnapPark is a cloud-based parking-violation reporting platform that lets citizens submit photographic evidence and have it analysed automatically by AI. The flow is simple: a citizen sees a vehicle parked illegally, photographs it, submits it through the web application, and the system analyses the image, stores the case and notifies the relevant parties.

The system is built as three independently deployed services — Authentication, Violation Analysis and Notification — behind a single API Gateway that receives all traffic from the browser-based front-end, a Next.js 14 web application. Each service owns its own PostgreSQL database and communicates with the others either through the gateway (synchronously) or through a RabbitMQ broker (asynchronously, for events).

There are two user roles. The default is **citizen**: anyone who registers and submits reports. The second is **admin**, assigned automatically at registration to any email listed in the `ADMIN_EMAILS` environment variable. Admins can view all cases and escalate them to the authority, whereas citizens see and manage only their own. Authorities and vehicle owners are not user accounts: authorities are reached through an outbound escalation workflow, and vehicle owners are notification recipients.

The requirements below were derived from the system as actually built; they describe what the application does.

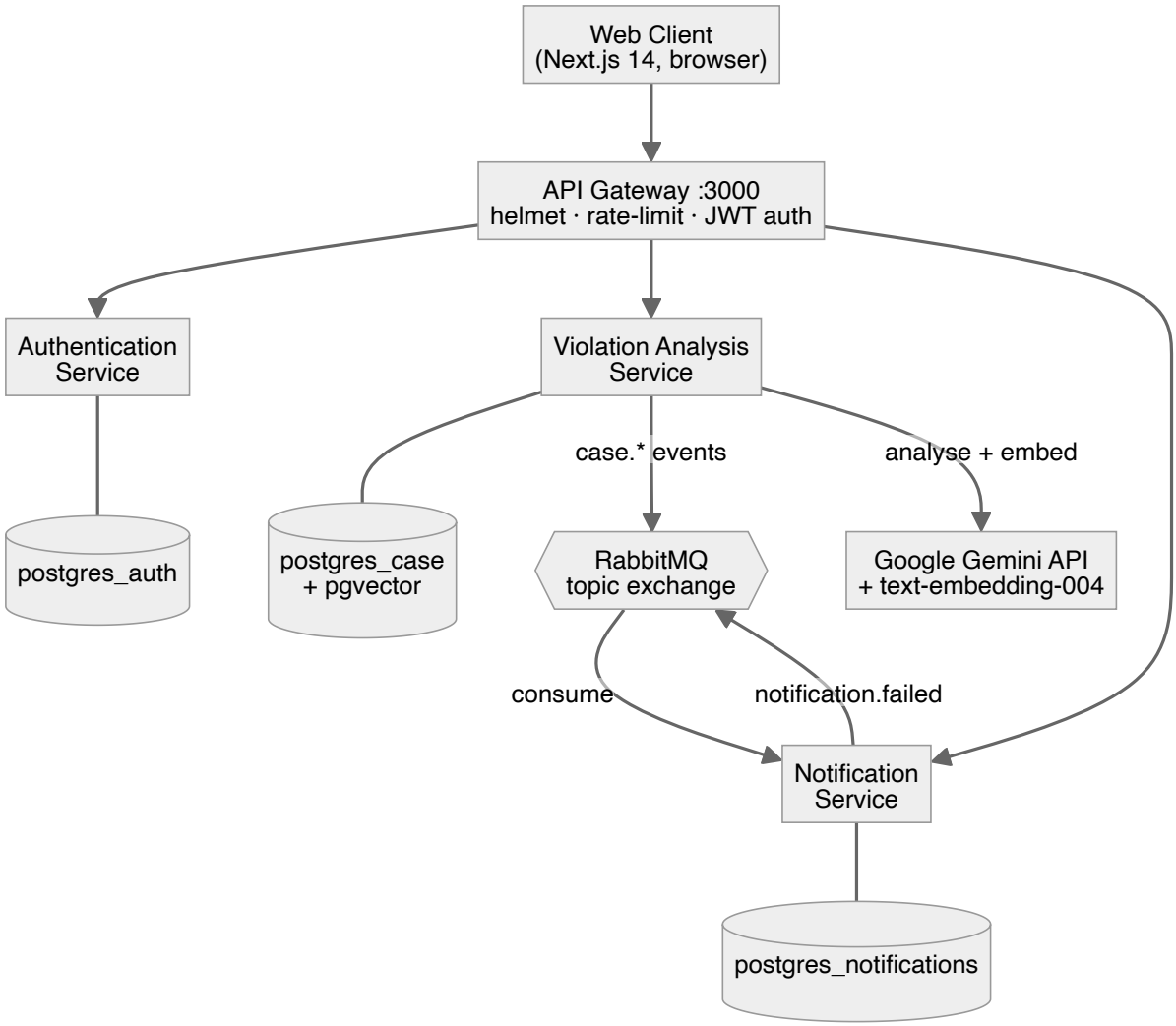

***Figure 3.1** — SnapPark high-level architecture — web client → API Gateway → Authentication / Violation Analysis / Notification services, each with its own PostgreSQL database, plus RabbitMQ and the Gemini API.*

3.2 Functional Requirements

- **FR1 — Registration and authentication.** Register, verify by emailed one-time password (OTP), then log in to receive a JWT access token and a refresh token; the refresh token rotates on use and is revoked on logout.
- **FR2 — Report submission.** An authenticated citizen submits one to five images with an optional licence plate; submissions over five images or of unsupported types are rejected before processing.
- **FR3 — AI-assisted analysis.** Uploaded images are sent to Gemini (`gemini-2.5-flash`), which returns a confirmation flag, violation type, confidence and explanation, stored with the case.
- **FR4 — Case lifecycle.** Cases move through `completed`, `reported_to_authority`, `resolved` and `cancelled`; a citizen may cancel their own case provided it has not

reached a terminal status such as completed or resolved; an admin may escalate or resolve. Every transition is audit-logged.

- **FR5 — Similar-case retrieval.** A user can retrieve semantically similar cases, computed from 768-dimensional `text-embedding-004` vectors stored via pgvector and searched with an HNSW index using cosine distance.
- **FR6 — Multi-channel notifications.** Case events are delivered through one or more of four channels — in-app, email (SMTP), SMS (Twilio) and push (Firebase) — selected by the recipient's stored preferences.
- **FR7 — History and statistics.** A citizen retrieves a paginated list of their own cases, filterable by status and date range, plus a per-user statistics breakdown by status.
- **FR8 — Admin escalation and oversight.** Admins (assigned via `ADMIN_EMAILS`) can view all cases and escalate or resolve any case, bypassing the citizen ownership check.

FR	Requirement	Priority	Implemented by
FR1	Registration & authentication (OTP, JWT, refresh rotation)	High	Authentication Service + Gateway
FR2	Report submission (1–5 images, optional plate)	High	Violation Analysis (via Gateway)
FR3	AI-assisted analysis (Gemini structured verdict)	High	Violation Analysis
FR4	Case lifecycle + audit log	High	Violation Analysis
FR5	Similar-case retrieval (pgvector / HNSW)	Medium	Violation Analysis
FR6	Multi-channel notifications (in-app, email, SMS, push)	High	Notification Service
FR7	History & per-user statistics	Medium	Violation Analysis
FR8	Admin escalation & oversight	Medium	Violation Analysis + Gateway

***Table 3.1** — Functional requirements FR1–FR8 with priority and the service that implements each.*

3.3 Non-Functional Requirements

- **NFR1 — Security.** All traffic passes through the gateway, which applies Helmet security headers and per-IP rate limiting and validates JWTs before forwarding; passwords are bcrypt-hashed; refresh tokens rotate on use and are revoked on logout.
- **NFR2 — Scalability.** Each deployable component — the three services and the gateway — has a Kubernetes Horizontal Pod Autoscaler so the cluster can scale them

independently on CPU utilisation.

- **NFR3 — Fault tolerance and consistency.** Case creation runs as a six-step orchestrated saga with compensating transactions executed in reverse on failure; saga state is persisted to the `sagas` table for inspection and recovery.
- **NFR4 — Loose coupling.** Services exchange events through a RabbitMQ topic exchange (`case.created`, `case.reported`, `case.resolved`, `case.cancelled`, `notification.failed`) rather than direct calls, so one failure does not block another.
- **NFR5 — Upload constraints.** A maximum of five images per report; multipart uploads are capped at 5 MB per image and JSON image bodies at 50 MB; over-limit or unsupported uploads are rejected before processing.
- **NFR6 — Vector-search performance.** Similarity queries use the HNSW index (`idx_cases_embedding_hnsw`, `vector_cosine_ops`), avoiding full table scans as the dataset grows.
- **NFR7 — Testability.** Each service has its own automated suite (Jest/Supertest or Vitest/Supertest), coverage is measured with Istanbul across all four IEEE-982 metrics, and each service runs in isolation via Docker Compose.
- **NFR8 — Environment parity.** A single Docker Compose file reproduces the full system locally (three services, gateway, three PostgreSQL instances, RabbitMQ, pgAdmin); the same system deploys to Kubernetes via the manifests in `deployment/kubernetes/`.

NFR	Requirement	Objective (§1.2)
NFR1	Security (gateway auth, Helmet, rate-limit, bcrypt, token rotation)	Security
NFR2	Scalability (per-component Horizontal Pod Autoscalers)	Scalability
NFR3	Fault tolerance & consistency (orchestrated saga + compensations)	High availability
NFR4	Loose coupling (RabbitMQ topic exchange)	High availability / Extensibility
NFR5	Upload constraints (≤5 images, 5 MB/image, type checks)	Security
NFR6	Vector-search performance (HNSW index, cosine distance)	Scalability
NFR7	Testability (per-service suites, Istanbul coverage)	Maintainability
NFR8	Environment parity (Docker Compose ↔ Kubernetes)	Maintainability

Table 3.2 *— Non-functional requirements NFR1–NFR8 mapped to the objectives in §1.2.*

Chapter 4 — Project Management

4.1 Software Development Process

A development process had to be chosen before any code was written. Three families were considered.

The **Waterfall** model runs requirements, design, implementation, testing and deployment as sequential, non-overlapping phases. It is simple to plan and document, but it assumes the requirements are fully known up front and produces no working software until late, which makes it unsuitable for a project where the behaviour of an AI vision model — and therefore parts of the design around it — could only be understood by experimentation.

The **Agile/Scrum** model is highly adaptive, organising work into short sprints with continual stakeholder feedback. Its ceremonies and the assumption of a team and a product owner fit a multi-person commercial setting better than a single student working to a fixed academic deadline with one supervisor.

The **Incremental** model sits between the two: the system is built as a sequence of increments, each of which passes through analysis, design, implementation, testing and evaluation, and each of which delivers a working, demonstrable slice of the whole. It keeps the per-increment discipline of waterfall while delivering working software early and allowing later increments to absorb the lessons of earlier ones.

SnapPark adopted the **Incremental** model. It matched both the project (a microservices system decomposes naturally into vertical slices that can be built and demonstrated one at a time) and the constraints (one developer, a fixed timeline, and a back-end whose behaviour had to be learned by building it). Four increments were defined, each ending in a working, tested deliverable, and each treated as a self-contained chapter (5–8) in this report.

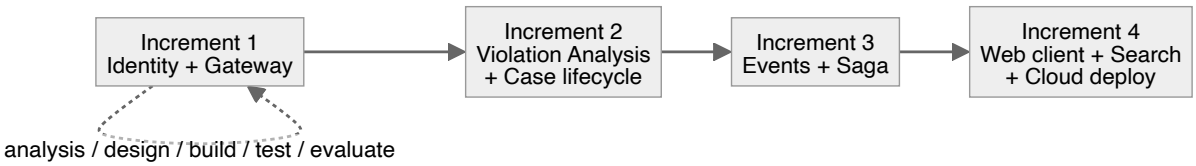

***Figure 4.1** — The incremental development model as applied to SnapPark's four increments.*

The four increments were:

1. **Identity and the API Gateway** — registration, login, JWT issuance and refresh rotation, and the gateway that fronts every request.
2. **Violation Analysis and the Case Lifecycle** — the Gemini-backed analysis pipeline, image validation, case persistence with per-service data ownership, the audit log and case management.
3. **Event-Driven Notifications and the Orchestrated Saga** — the multi-channel Notification Service, RabbitMQ messaging, and the orchestrated saga that ties case

creation together with compensations.

4. **Web Client, Semantic Search and Cloud Deployment** — the Next.js web application (including OTP onboarding and the admin panel), pgvector-based similar-case retrieval, and Kubernetes deployment.

The version-control history reflects this progression, with feature branches such as `feature/saga-orchestration`, `feature/otp-auth-flows`, `feature/polyglot-pgvector` and `feature/kubernetes-deployment` merged into `develop` as each slice was completed, following a Gitflow branching model.

4.2 Risk Management

Risks were identified at the outset and tracked throughout. The main ones and their mitigations were:

- **Dependence on a third-party AI provider.** The analysis pipeline depends on the Gemini API, which could change, rate-limit or fail. *Mitigation:* the verdict is treated as an opinion rather than a fact; the analysis is isolated behind a single module (`gemini.js`) so the provider can be swapped; and a deterministic local fallback for embeddings keeps the pipeline runnable without a live key during development and testing.
- **Distributed-transaction failure leaving partial data.** A multi-step case creation could fail halfway and leave orphaned rows. *Mitigation:* the orchestrated saga with compensations and persisted state (NFR3).
- **Cost overrun on paid APIs.** Per-call charges for vision and embedding could accumulate. *Mitigation:* a single combined analysis call per submission, image pre-validation to reject unsuitable uploads before they reach the model, and a single vendor for both vision and embeddings to simplify budgeting.
- **Scope creep.** A civic platform invites endless features. *Mitigation:* the increment plan fixed a deliverable per increment, and features outside the current increment were deferred.
- **Environment and reproducibility problems.** "Works on my machine" is a real risk in a multi-service system. *Mitigation:* full containerisation, a single Docker Compose file for local parity, and Kubernetes manifests for production (NFR8).
- **Data security and credential leakage.** Secrets and user credentials must not leak. *Mitigation:* bcrypt-hashed passwords, JWTs validated at the gateway, and secrets kept in environment variables and Kubernetes Secrets rather than in the codebase.

Risk	Likelihood	Impact	Mitigation
Dependence on a third-party AI provider	Medium	High	Verdict treated as opinion; analysis isolated behind `gemini.js`; deterministic local fallback
Distributed-transaction failure leaving partial data	Medium	High	Orchestrated saga with compensations and persisted state (NFR3)
Cost overrun on paid APIs	Medium	Medium	Single combined analysis call; image pre-validation; single vendor for vision + embeddings
Scope creep	High	Medium	Fixed deliverable per increment; out-of-scope features deferred
Environment / reproducibility problems	Medium	Medium	Full containerisation; one Docker Compose file; Kubernetes manifests
Data security & credential leakage	Low	High	bcrypt-hashed passwords; JWT validated at gateway; secrets in env / Kubernetes Secrets

***Table 4.1** — Risk register — likelihood, impact and mitigation for each identified risk.*

Chapter 5 — Increment 1: Identity and the API Gateway

5.1 Analysis

The first increment had to establish the spine of the system: a way for users to exist, authenticate, and have every subsequent request mediated by a single, secured entry point. Without identity and a gateway, none of the later increments — submitting cases, receiving notifications, viewing one's own history — could be built safely. The increment therefore covers FR1 (registration and authentication) and the security and maintainability foundations of NFR1.

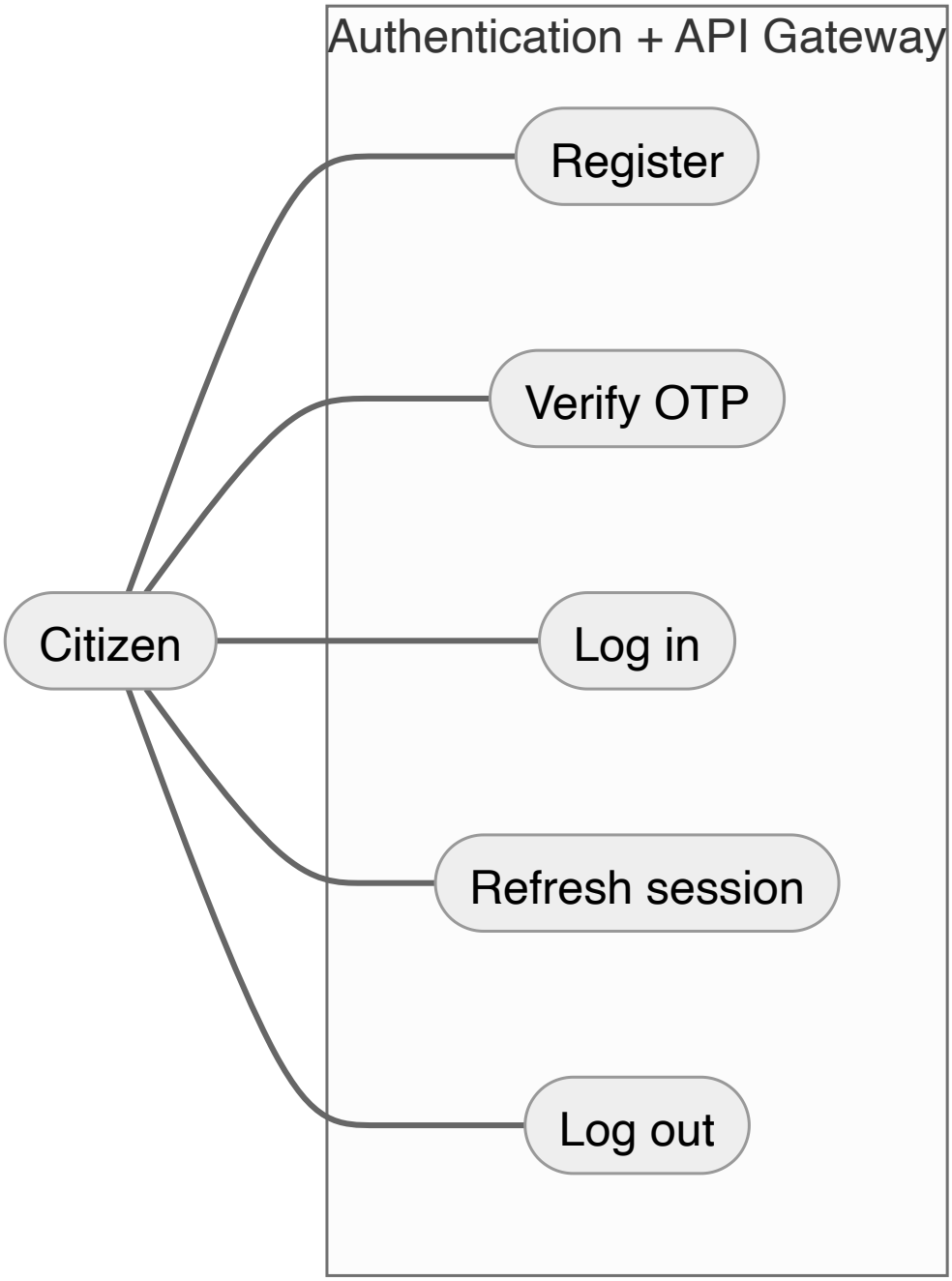

***Figure 5.1** — Use-case diagram for Increment 1 — Register, Verify, Log in, Refresh, Log out.*

5.2 Design

Two components were designed. The **Authentication Service** owns user identity and credentials in its own `snappark_auth` database, following Database-per-Service (§2.3). The **API Gateway** is the single entry point, validating tokens and routing to downstream services. Authentication is stateless via JWTs (§2.8.5): the gateway validates an access token on each request and forwards only the user's identity downstream in `X-User-Id` and `X-User-Role` headers, so downstream services never see the token or query the auth database. Refresh tokens are persisted so they can be rotated and revoked.

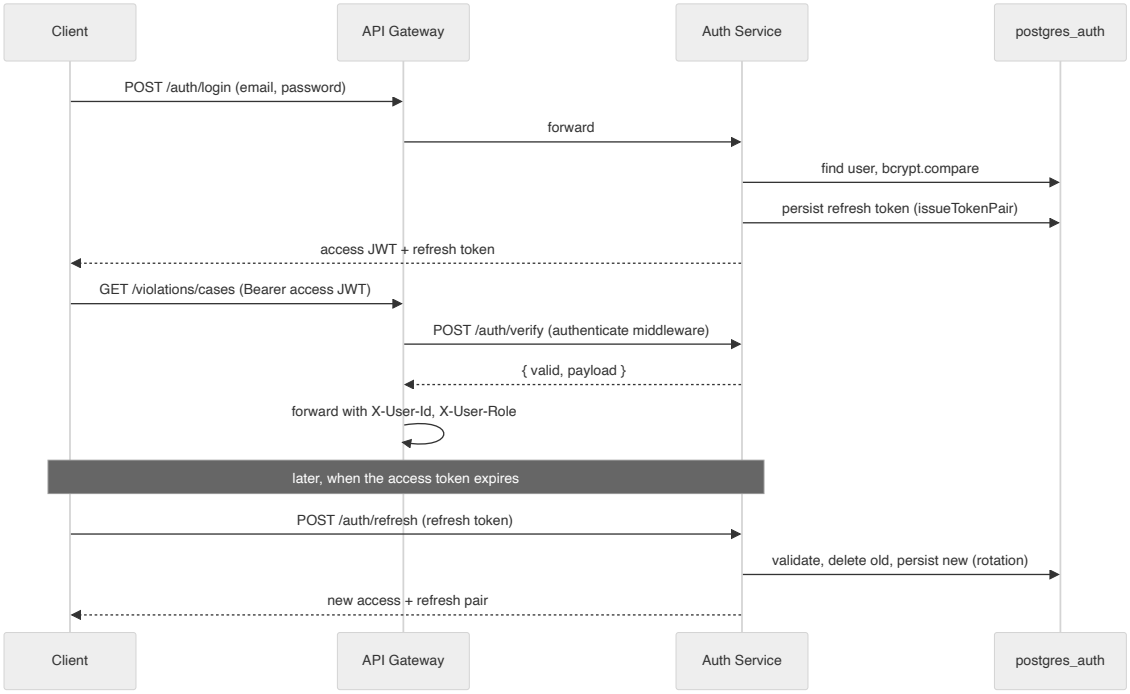

***Figure 5.2** — Sequence diagram — login, token issuance, an authenticated request through the gateway, and refresh.*

5.3 Implementation

The Authentication Service is an Express application. On registration the password is hashed with bcrypt before storage (`helpers.js`), and a record is created in the `users` table whose `role` defaults to `citizen`; if the email matches the `ADMIN_EMAILS` allowlist, `ensureAdminRole()` promotes the account to `admin`. The token pair is issued by `issueTokenPair()`, which signs an access token and a refresh token and persists the refresh token to a `refresh_tokens` table with its expiry. Login (`/auth/login`) verifies the password with `bcrypt.compare`. Refresh (`/auth/refresh`) validates the presented refresh token against the stored row, deletes it and issues a new pair — rotation — so a stolen token cannot be reused after the legitimate client refreshes. Logout (`/auth/logout`) deletes the refresh token, and a password reset revokes all of a user's refresh tokens at once.

The API Gateway (`api-gateway/src/index.js`) applies `helmet()` for security headers, a 1 MB JSON body limit, and an `express-rate-limit` limiter on every route. Protected routes pass through an `authenticate` middleware that validates the JWT before the request is proxied to the relevant backend.

```
// Rate limiter — applies to every route
const windowMs    = Number(process.env.RATE_LIMIT_WINDOW_MS || 15 * 60 * 1000); // 15
        min
const maxRequests = Number(process.env.RATE_LIMIT_MAX_REQUESTS || 100);
app.use(rateLimit({
  windowMs,
```

```
  max: maxRequests,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too many requests. Please try again later.' },
}));

// ── Authentication Middleware ──────────────────────────────────────
//
// Contacts the Authentication Service to verify the caller's JWT.
// On success the decoded payload (sub, email) is attached to req.user
// so downstream handlers know which user is making the request.
// On failure the request is rejected immediately — it never reaches any
// core microservice.  This matches the architectural diagram:
//
//   Client → API Gateway → Auth Service (verify) → ✅ route to service
//                                                → ❌ reject with 401

const authenticate = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
     return res.status(401).json({ error: 'Missing or malformed Authorization header.'
        });
  }

  try {
    const { data } = await axios.post(
      `${AUTH_SERVICE_URL}/auth/verify`,
      {},
      { headers: { Authorization: authHeader }, timeout: 5000 },
    );

    if (!data.valid) {
      return res.status(401).json({ error: 'Invalid or expired token.' });
    }

    // Attach user info for downstream services
    req.user = data.payload; // { sub, email, iat, exp }
    next();
  } catch (err) {
    // If the auth service itself returned 401 with details, pass them through
    if (err.response?.status === 401) {
      return res.status(401).json(err.response.data);
```

```
    }
    console.error('[gateway] Auth service error:', err.message);
    return res.status(503).json({ error: 'Authentication service unavailable.' });
  }
};
```

Listing 5.1 *— The gateway's authenticate middleware and rate limiter.*

```
/** Sign a short-lived access token. The role claim drives admin-only routes. */
export const createAccessToken = (user, secret, expiresIn) =>
  jwt.sign(
        { sub: user.id, email: user.email, role: user.role || 'citizen', jti:
        crypto.randomUUID() },
    secret,
    { expiresIn },
  );

/** Sign a long-lived refresh token */
export const createRefreshToken = (user, secret, expiresIn) =>
   jwt.sign({ sub: user.id, type: 'refresh', jti: crypto.randomUUID() }, secret, {
        expiresIn });

// ……

const issueTokenPair = async (user) => {
  const accessToken  = createAccessToken(user, JWT_SECRET, TOKEN_EXPIRY);
  const refreshToken = createRefreshToken(user, JWT_REFRESH_SECRET, REFRESH_EXPIRY);

  const { exp } = jwt.decode(refreshToken);
  await query(
    `INSERT INTO refresh_tokens (user_id, token, expires_at)
     VALUES ($1, $2, to_timestamp($3))`,
    [user.id, refreshToken, exp]
  );

  return { accessToken, refreshToken };
};
```

Listing 5.2 *— issueTokenPair — access/refresh signing and refresh-token persistence.*

5.4 Testing

The Authentication Service is tested with Jest and Supertest. Unit tests cover the pure helpers (hashing, token creation), and integration tests exercise the route handlers against a live PostgreSQL database, covering registration, login, refresh rotation, logout, profile and password changes, and the error and edge cases. The gateway is tested with Vitest and Supertest, covering proxying, the authenticate middleware and the error paths.

```
$ cd services/api-gateway && npx vitest run
 Test Files  2 passed (2)
      Tests  47 passed (47)

# Authentication Service (Jest, requires a live PostgreSQL instance):
# full unit + integration suite ≈ 77% statement / 70% decision-condition
# (see docs/coverage-methodology.md)
```

Figure 5.3 *— Test results for Increment 1 — authentication and gateway suites passing.*

5.5 Evaluation

The increment delivered a working identity foundation: a user can register, verify, log in, make authenticated requests through the gateway, and have their session rotated and revoked. The objectives of security (every request authenticated at the gateway, no plain-text passwords, revocable short-lived tokens) and maintainability (two small, independently testable services) were met for this slice. The main limitation at this stage was that email verification used a link-based flow that was later replaced by OTP in Increment 4 for a smoother web experience.

Chapter 6 — Increment 2: Violation Analysis and the Case Lifecycle

6.1 Analysis

With identity in place, the second increment built the core purpose of the system: turning a submitted photograph into a stored, analysed case. It covers FR2 (submission), FR3 (AI analysis), FR4 (lifecycle and audit) and FR7 (history and statistics), and exercises the data-management decisions of §2.3 and the AI decision of §2.6.

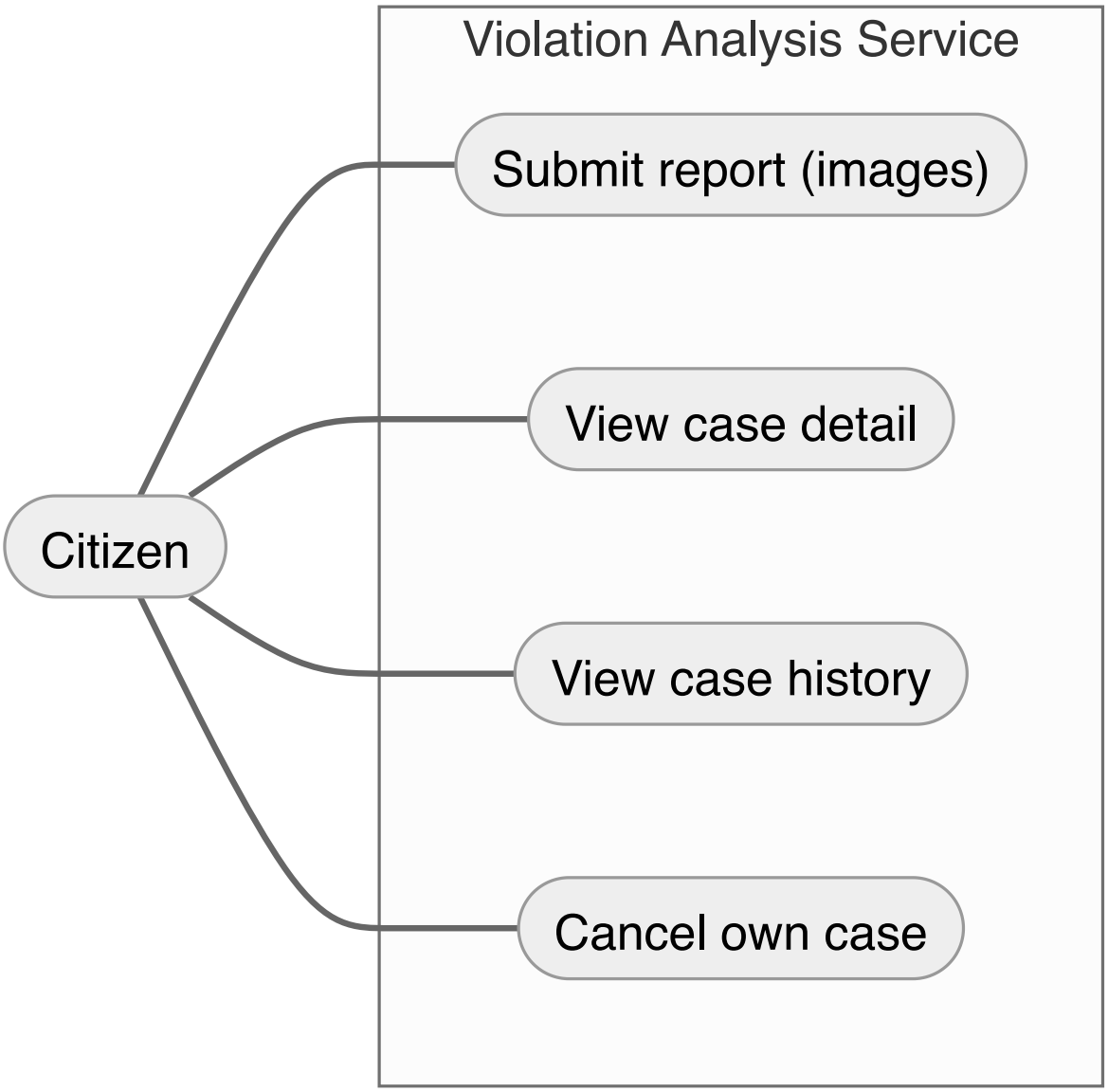

Figure 6.1 *— Use-case diagram — Submit report, View case, View history, Cancel case.*

6.2 Design

The **Violation Analysis Service** owns the `snappark_case` database. A case is created through a pipeline: validate the upload, obtain a verdict from Gemini, persist the case and its images, and write an audit entry. Images are accepted either as multipart uploads or as Base64 JSON, with limits enforced before any processing (one to five images, 5 MB per image, accepted MIME types `image/jpeg`, `image/png`, `image/webp`). The case carries a status (`completed`, `reported_to_authority`, `resolved`, `cancelled`) and the structured verdict fields. An append-only audit log records every state change for auditability.

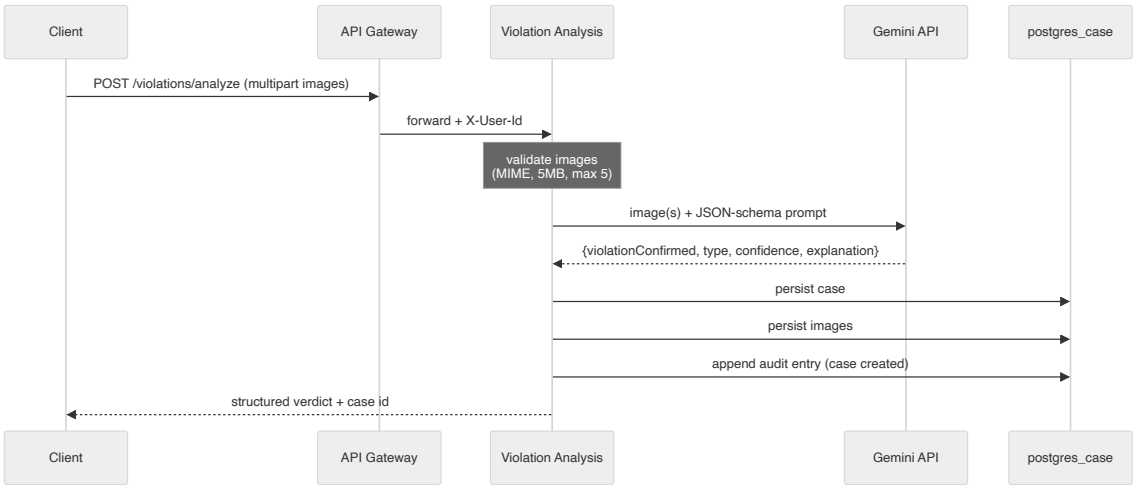

***Figure 6.2** — Sequence diagram — submission → validation → Gemini analysis → case + image persistence → audit entry → response.*

6.3 Implementation

The `POST /violations/analyze` route uses multer (memory storage) with a 5 MB per-file limit and a MIME-type `fileFilter`. After parsing the upload (and an optional, normalised licence plate), it validates the images, calls `analyseImage` / `analyseMultipleImages` in `gemini.js`, and persists the result. The Gemini module sends each image as Base64 `inlineData` with a schema-bearing prompt and parses the response into `{violationConfirmed, violationType, confidence, explanation}`. Cases are read back through `GET /violations/cases`, which supports filtering by `userId`, `status` and a `from` / `to` date range with pagination, and `GET /violations/stats/:userId`, which returns counts per status. Cancellation (`DELETE /violations/:id`) enforces ownership (non-admins may only act on their own cases) and refuses to cancel a `completed` or already-`cancelled` case, writing a `CaseCancelled` audit event on success. A scheduled cleanup job discards unprocessed images past a time threshold.

```
export const analyseImage = async (base64Data, mimeType) => {
  try {
    const imagePart = { inlineData: { data: base64Data, mimeType } };
```

```
    const result = await model.generateContent([SINGLE_IMAGE_PROMPT, imagePart]);
    return parseResponse(result.response.text().trim());
  } catch (err) {
    if (DEMO_MODE && isGeminiApiError(err)) {
              console.warn('[gemini]  API  error  –  returning  demo  verdict:',
        err.message?.slice(0, 200));
      return mockVerdict(base64Data);
    }
    throw err;
  }
};

// ……

export const analyseMultipleImages = async (images) => {
  try {
    const imageParts = images.map((img) => ({
      inlineData: { data: img.base64, mimeType: img.mimeType },
    }));
    const result = await model.generateContent([MULTI_IMAGE_PROMPT, ...imageParts]);
    return parseResponse(result.response.text().trim());
  } catch (err) {
    if (DEMO_MODE && isGeminiApiError(err)) {
          console.warn('[gemini]  Quota/auth  error  –  returning  demo  verdict.  Set
        GEMINI_DEMO_MODE=false to disable.');
      return mockVerdict(images[0]?.base64 || '');
    }
    throw err;
  }
};
```

Listing 6.1 *— gemini.js — prompt construction and JSON-schema parsing of the verdict.*

```
app.get('/violations/cases', async (req, res) => {
  try {
    const { userId, status, from, to, limit = '50', offset = '0' } = req.query;
    const maxLimit = Math.min(Number(limit) || 50, 200);
    const skip     = Math.max(Number(offset) || 0, 0);

    const conditions = [];
    const params     = [];
    let paramIndex   = 1;
```

```
    if (userId) {
      conditions.push(`user_id = $${paramIndex++}`);
      params.push(userId);
    }
    if (status) {
      conditions.push(`status = $${paramIndex++}`);
      params.push(status);
    }
    if (from) {
      conditions.push(`created_at >= $${paramIndex++}`);
      params.push(from);
    }
    if (to) {
      conditions.push(`created_at <= $${paramIndex++}`);
      params.push(to);
    }

    const where = conditions.length > 0 ? `WHERE ${conditions.join(' AND ')}` : '';

    // Get total count for pagination metadata
     const countResult = await query(`SELECT COUNT(*) as total FROM cases ${where}`,
        params);
    const total = Number(countResult.rows[0].total);

    // Get paginated results
    const result = await query(
       `SELECT * FROM cases ${where} ORDER BY created_at DESC LIMIT $${paramIndex++}
        OFFSET $${paramIndex}`,
      [...params, maxLimit, skip]
    );

    return res.status(200).json({
      cases:  result.rows,
      count:  result.rowCount,
      total,
      limit:  maxLimit,
      offset: skip,
    });
```

***Listing 6.2** — The case-listing query with status and date-range filtering and pagination.*

6.4 Testing

The service is tested with Vitest and Supertest, covering the analyze route (including the validation and size/MIME rejections), the database layer, the Gemini integration (with the model mocked), the image validator and the cleanup job, plus branch-level edge cases. Measured coverage for this service is high (see Chapter 9).

```
$ cd services/violation-analysis-service && npx vitest run
 Test Files  13 passed (13)
      Tests  155 passed (155)
```

Figure 6.3 *— Test results for Increment 2 — violation-analysis suite passing with coverage summary.*

6.5 Evaluation

The increment delivered the system's central capability: a citizen can submit images and receive a structured, explained verdict, and the case is stored, listable, filterable and auditable. The Database-per-Service decision held up — the service owns its data and exposes it only through its API — and routing analysis through a single, swappable Gemini module kept the third-party dependency contained, addressing the provider-risk identified in Chapter 4. The lifecycle and audit log satisfy the auditability objective for case state.

Chapter 7 — Increment 3: Event-Driven Notifications and the Orchestrated Saga

7.1 Analysis

The third increment addressed two related concerns: telling people what happened to a case, and guaranteeing that the multi-step creation of a case is consistent even when a step fails. It covers FR6 (notifications) and NFR3 (fault tolerance) and NFR4 (loose coupling), and applies the communication decision of §2.4 and the saga decision of §2.3.1–2.3.2.

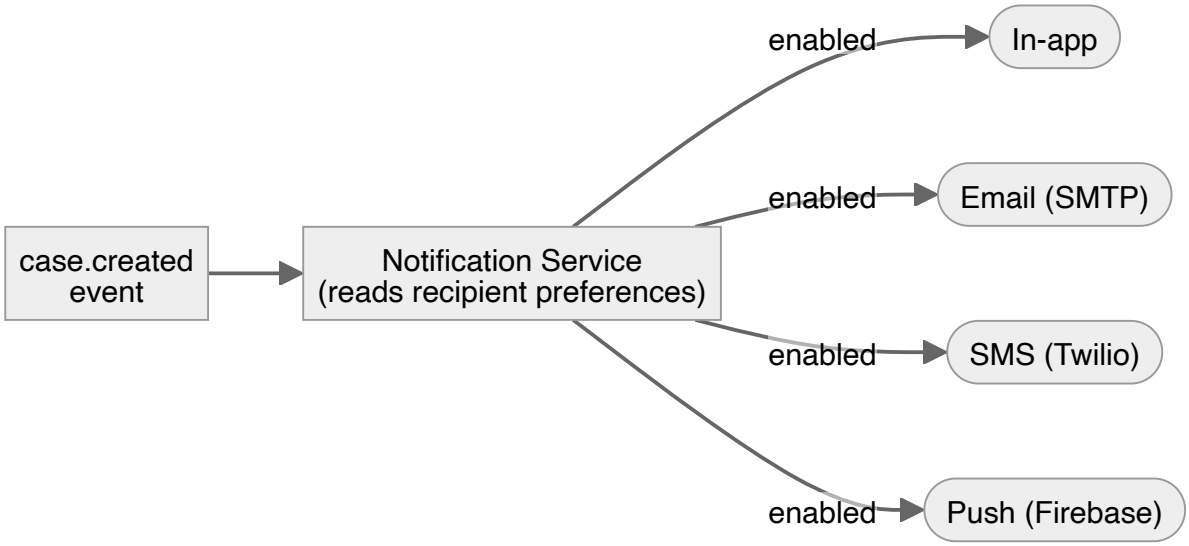

***Figure 7.1** — Use-case / event diagram — a case event fanning out to enabled notification channels.*

7.2 Design

The **Notification Service** owns `snappark_notifications` and consumes case events from a RabbitMQ topic exchange, decoupling it entirely from the analysis pipeline: a failed or slow notification cannot block case creation (high-availability objective). Delivery is designed around interchangeable channel implementations (in-app, email, SMS, push) selected per recipient from stored preferences (extensibility objective — a new channel is a new implementation, not a change to existing ones). Case creation itself is designed as an orchestrated saga so its six steps either all succeed or are cleanly compensated.

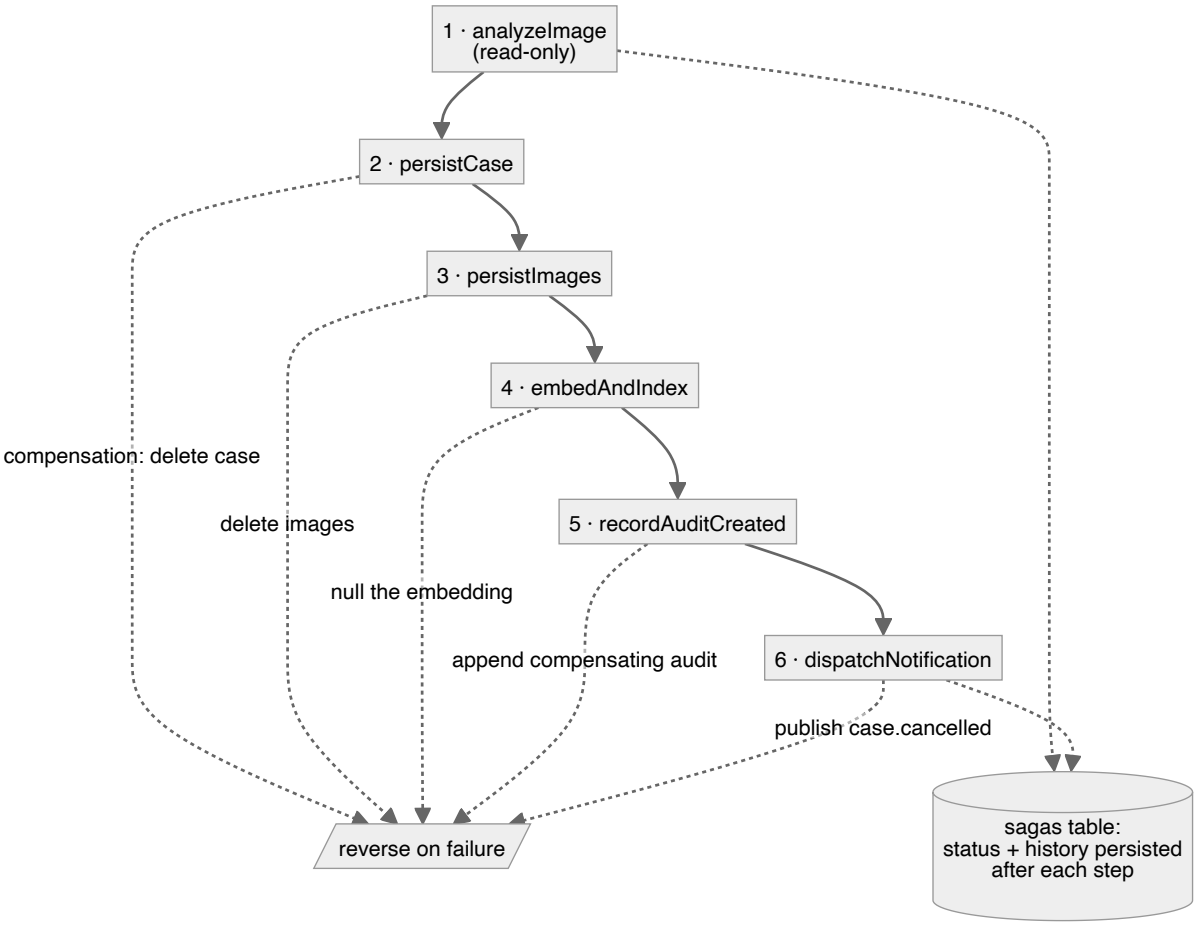

***Figure 7.2** — The orchestrated case-creation saga — six steps with their compensations, and persisted saga state.*

7.3 Implementation

The saga (`saga/caseCreationSaga.js`) defines six steps: `analyzeImage` (read-only, no compensation), `persistCase` (↩ delete case), `persistImages` (↩ delete images), `embedAndIndex` (↩ null the embedding), `recordAuditCreated` (↩ append a compensating audit entry) and `dispatchNotification` (↩ publish `case.cancelled`). The coordinator (`saga/coordinator.js`) runs the steps in order, persisting saga status and history to the `sagas` table; on a step failure it runs the completed steps' compensations in reverse. Events are published to a `'topic'` exchange in `rabbitmq.js` as `case.created`, `case.reported`, `case.resolved` and `case.cancelled`.

The Notification Service's dispatcher (`dispatcher.js`) loads the recipient's preferences, determines the enabled channels (skipping channels that are disabled, unregistered, or lack a delivery address), and fans out to them concurrently with `Promise.allSettled` so one failing channel does not block the others. If every enabled channel fails, it publishes `notification.failed` back to the exchange, which the saga consumes to record the failure — closing the compensation loop. The channels are implemented behind a common `BaseChannel` abstraction, so adding a channel means adding one class and registering it.

```
for (const step of steps) {
  await updateSaga(sagaId, { current_step: step.name });
  await appendHistory(sagaId, {
    step:   step.name,
    action: 'execute',
    status: 'started',
    at:     new Date().toISOString(),
  });

  try {
    const partial = await step.execute(context);
    context = safeMerge(context, partial);

    // The context column is the canonical record of saga progress —
    // refresh it after every successful step.
    await updateSaga(sagaId, { context });
    await appendHistory(sagaId, {
      step:   step.name,
      action: 'execute',
      status: 'succeeded',
      at:     new Date().toISOString(),
    });

    completed.push(step);
  } catch (err) {
    await appendHistory(sagaId, {
      step:   step.name,
      action: 'execute',
      status: 'failed',
      error:  err.message,
      at:     new Date().toISOString(),
    });

    await updateSaga(sagaId, {
      status:       SagaStatus.COMPENSATING,
      error:        `${step.name}: ${err.message}`,
    });

    const compensationOk = await runCompensations({
      sagaId,
      completed,
```

```
        context,
      });

      await updateSaga(sagaId, {
        status: compensationOk ? SagaStatus.COMPENSATED : SagaStatus.FAILED,
      });

          const compositeErr = new Error(`Saga '${sagaType}' failed at step
        '${step.name}': ${err.message}`);
      compositeErr.sagaId       = sagaId;
      compositeErr.failedStep   = step.name;
      compositeErr.cause        = err;
      compositeErr.compensated  = compensationOk;
      throw compositeErr;
    }
  }

// ……

const runCompensations = async ({ sagaId, completed, context }) => {
  // Run compensations in reverse order so resources are released in the
  // opposite order they were acquired (LIFO).
  for (const step of [...completed].reverse()) {
    if (typeof step.compensate !== 'function') continue;

    await appendHistory(sagaId, {
      step:   step.name,
      action: 'compensate',
      status: 'started',
      at:     new Date().toISOString(),
    });

    try {
      await step.compensate(context);
      await appendHistory(sagaId, {
        step:   step.name,
        action: 'compensate',
```

***Listing 7.1** — The saga coordinator running steps and compensations with persisted state.*

```
  // 3. Determine which channels to dispatch to
  const enabledChannels = ['in_app', 'sms', 'email', 'push'].filter((ch) => {
```

```
    if (!prefs[ch]) return false;              // user disabled this channel
    if (!channels.has(ch)) return false;       // channel not registered (env missing)

    const addrField = ADDRESS_FIELD[ch];
    if (addrField && !prefs[addrField]) {      // external channel but no address on file
      console.warn(`[dispatcher] ${ch} enabled for user ${event.userId} but no ${addrField} configured — skipping`);
      return false;
    }

    return true;
  });

  // 4. Fan out to all enabled channels concurrently
  const results = await Promise.allSettled(
    enabledChannels.map(async (ch) => {
      const provider = channels.get(ch);
      const addrField = ADDRESS_FIELD[ch];
      const to = addrField ? prefs[addrField] : undefined;

      const result = await provider.send({
        to,
        caseId: event.id,
        userId: event.userId,
        subject,
        message,
        metadata,
      });

      // Log the delivery attempt
      await insertDeliveryLog({
        notificationId: result.providerResponse?.notificationId || null,
        caseId: event.id,
        userId: event.userId,
        channel: ch,
        status: result.success ? 'sent' : 'failed',
        providerResponse: result.providerResponse || null,
        errorMessage: result.error || null,
      });

      return { channel: ch, success: result.success, error: result.error };
    })
```

```
  );

  // 5. Normalise allSettled results
  const outcomes = results.map((r, i) => {
    if (r.status === 'fulfilled') return r.value;
    return { channel: enabledChannels[i], success: false, error: r.reason?.message };
  });

  // 6. Saga callback: if every enabled channel failed, publish
  //    notification.failed so the originating saga can record the failure
  //    and tag the case. We deliberately fire ONLY on full failure —
  //    a partial success (e.g. in-app worked but email bounced) means
  //    the user did get notified, so the saga doesn't need to act.
  //
  //    `event.sagaId` is the link back to the case-creation saga; if
  //    absent (e.g. the event came from a non-saga source), we still
  //    publish but the listener treats sagaId-less payloads as "no-op".
  if (outcomes.length > 0 && outcomes.every((o) => !o.success)) {
    publishNotificationFailed({
      sagaId:  event.sagaId ?? null,
      caseId:  event.id,
      userId:  event.userId,
      channel: outcomes.map((o) => o.channel).join(','),
        error:     outcomes.map((o)  =>  o.error).filter(Boolean).join('; ')  ||  'all
      channels failed',
    });
  }
```

***Listing 7.2** — The multi-channel dispatcher with concurrent, independent delivery.*

7.4 Testing

The Notification Service is tested with Vitest and Supertest across the routes, the channel registry and implementations, and the dispatcher (including the all-channels-failed branch). The saga is tested through its coordinator and every compensation path, and through the listeners that consume `notification.failed`. Both services report high coverage (Chapter 9).

```
$ cd services/notification-service && npx vitest run
 Test Files  7 passed (7)
      Tests  81 passed (81)
```

```
# Saga-coordinator and compensation tests run inside the
# violation-analysis suite above (13 files / 155 tests passing).
```

Figure 7.3 *— Test results for Increment 3 — notification and saga suites passing.*

7.5 Evaluation

The increment met the loose-coupling and fault-tolerance objectives directly. Notifications are fully decoupled through the topic exchange, so the analysis pipeline is unaffected by notification failures, and the choice of a topic exchange leaves room for new event consumers without touching publishers (extensibility). The orchestrated saga, with compensations and persisted state, ensures that a failure midway through case creation does not leave partial data — the concrete answer to the distributed-transaction risk from Chapter 4. Choosing orchestration over choreography paid off precisely because the workflow is now a single, inspectable artefact.

Chapter 8 — Increment 4: Web Client, Semantic Search and Cloud Deployment

8.1 Analysis

The final increment made the system usable and deployable, and added its most sophisticated retrieval feature. It covers FR1's OTP onboarding, FR5 (similar-case retrieval), FR8 (admin oversight) and NFR2/NFR6/NFR8 (scalability, vector-search performance and environment parity), applying the front-end, semantic-similarity and deployment decisions of §2.8.3, §2.7 and §2.5.

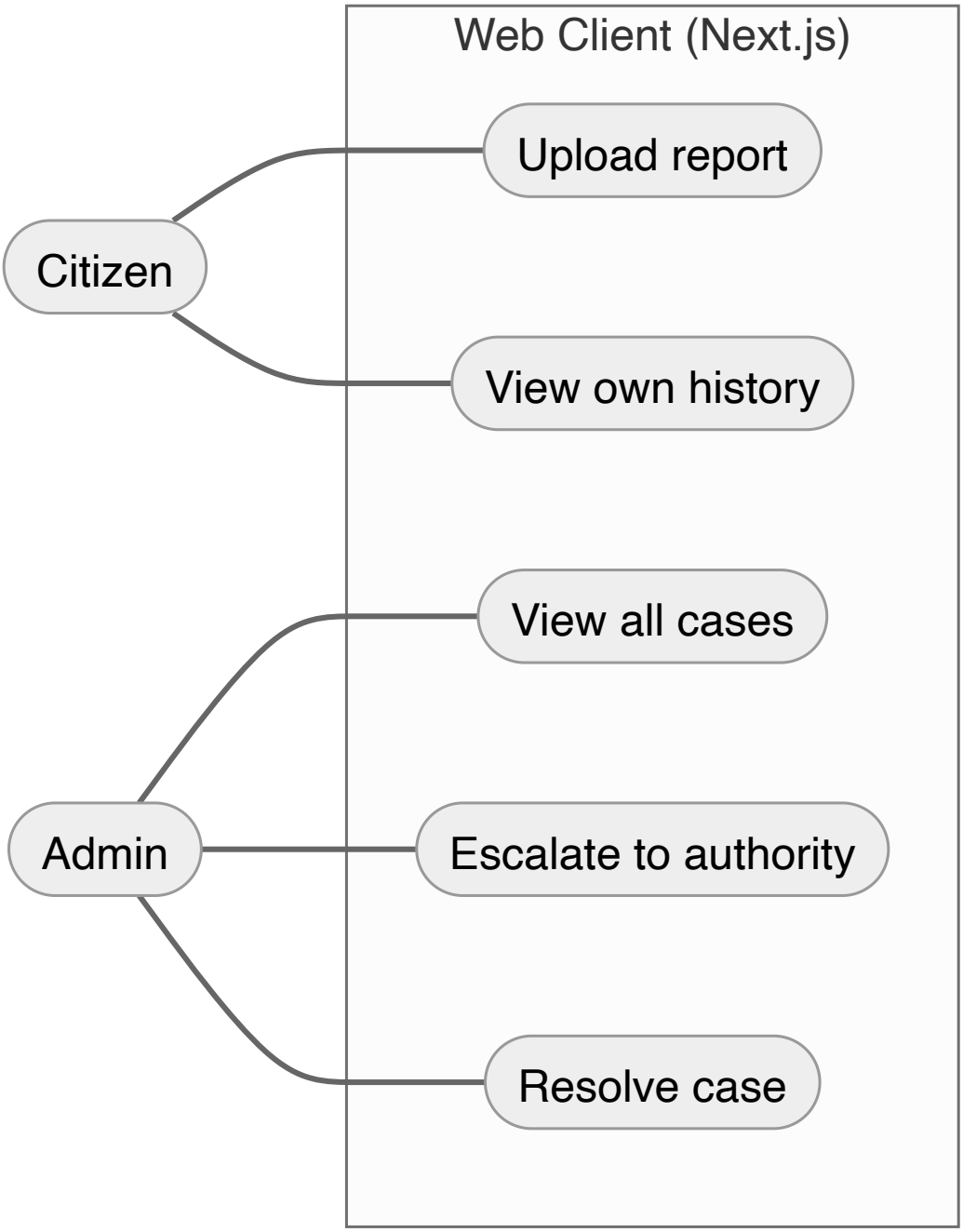

***Figure 8.1** — Use-case diagram — citizen upload/history and admin all-cases/escalation flows in the web client.*

8.2 Design

The front-end is a Next.js 14 App Router application styled with Tailwind, with route groups separating the authentication screens (login, register, OTP verification, password reset) from the authenticated application (dashboard, upload, case detail, notifications, settings, admin). The admin panel reuses citizen pages with elevated visibility rather than a separate UI. For retrieval, the verdict text of each case is embedded and stored in the same `cases` row via

pgvector, so similar cases can be found with a single cosine-distance query over an HNSW index. For production, every component is described as a Kubernetes resource with autoscaling.

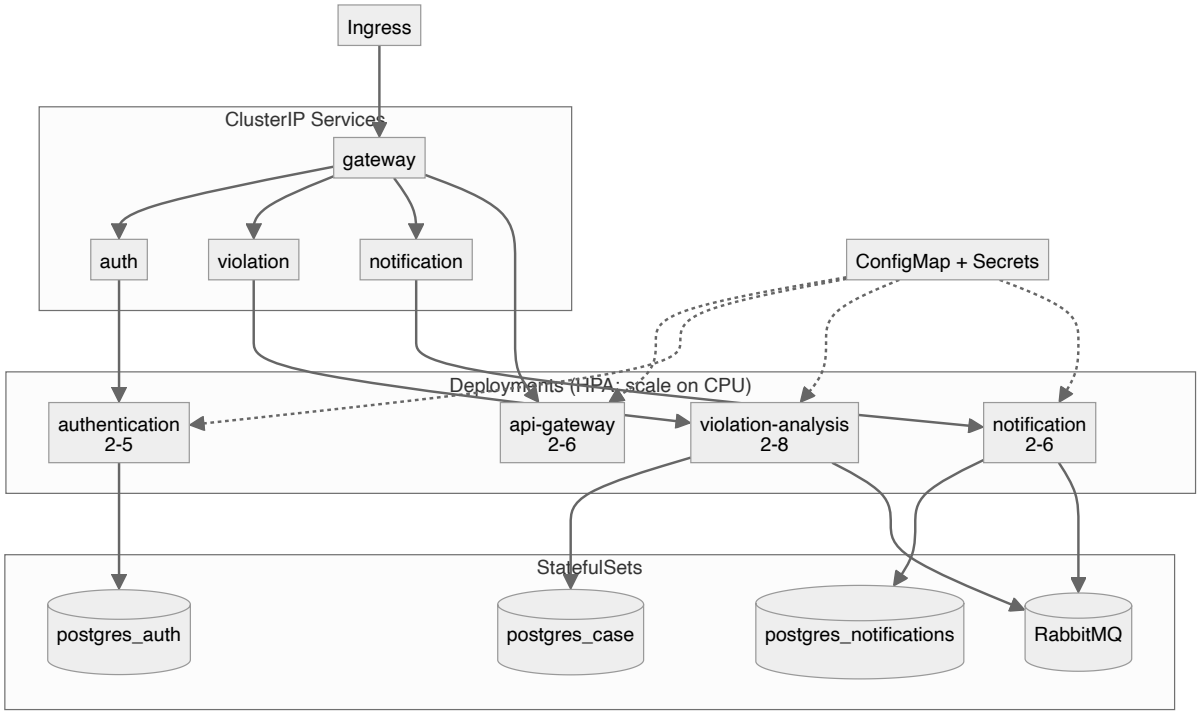

***Figure 8.2** — Kubernetes deployment topology — deployments, services, ingress, config maps, secrets, HPAs and StatefulSets.*

8.3 Implementation

The web client provides registration with first/last name, OTP verification (replacing the earlier link-based flow), password reset, image upload with an optional licence plate and location, a paginated case list, a case-detail page with a “Similar cases” panel, a notifications page and a settings page for channel preferences. Admin-only controls — the all-cases view and the report-to-authority action — are gated on the `admin` role, and the gateway forwards `X-User-Role` so the backend enforces the same rule.

Semantic search is implemented by the saga’s `embedAndIndex` step: `embeddings.js` computes a 768-dimensional vector with `text-embedding-004` (falling back to a deterministic local embedding when no API key is present, so the pipeline runs in development), and the vector is written to the embedding column. `GET /violations/:id/similar` returns the nearest cases using the `<=>` cosine operator over `idx_cases_embedding_hnsw`. Deployment is provided as a Docker Compose file for local parity (the three services, the gateway, three PostgreSQL instances, RabbitMQ and pgAdmin) and a full Kubernetes manifest set including a Horizontal Pod Autoscaler for each of the three services and the gateway.

```
      IF NOT EXISTS (SELECT 1 FROM information_schema.columns
                     WHERE table_name = 'cases' AND column_name = 'embedding') THEN
        ALTER TABLE cases ADD COLUMN embedding vector(${EMBEDDING_DIM});
      END IF;
    END
    $$;

    CREATE INDEX IF NOT EXISTS idx_cases_embedding_hnsw
      ON cases USING hnsw (embedding vector_cosine_ops);
  `);

// ……

  name: 'embedAndIndex',
  execute: async (ctx) => {
    const text = buildEmbeddingInput({
      violationType: ctx.savedCase.violation_type,
      explanation:   ctx.savedCase.explanation,
      licensePlate:  ctx.savedCase.license_plate,
    });
    if (!text) {
      // Nothing useful to embed (e.g. analysis returned no explanation
      // and no plate was supplied). Skip silently — the case row still
      // exists and the embedding column remains NULL, which makes the
      // case invisible to similarity search but preserves all other data.
      return { embedded: false };
    }
    const embedding = await generateEmbedding(text);
    await setCaseEmbedding(ctx.savedCase.id, embedding);
    return { embedded: true, embeddingLength: embedding.length };
  },

// ……

export const findSimilarCases = async (caseId, limit = 5) => {
  const r = await query(
    `SELECT c.id, c.user_id, c.status, c.violation_confirmed, c.violation_type,
            c.confidence, c.explanation, c.license_plate, c.created_at,
            c.embedding <=> src.embedding AS distance
       FROM cases c, cases src
      WHERE src.id = $1
```

```
        AND src.embedding IS NOT NULL
        AND c.id <> src.id
        AND c.embedding IS NOT NULL
      ORDER BY distance ASC
      LIMIT $2`,
    [caseId, limit]
  );
  return r.rows;
};
```

***Listing 8.1** — The embedAndIndex saga step and the HNSW similarity query.*

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: violation-analysis-service
  namespace: snappark
  labels:
    app: violation-analysis-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: violation-analysis-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: violation-analysis-service
    spec:
      containers:
        - name: violation-analysis-service
          image: snappark/violation-analysis-service:1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 3002
              name: http
          env:
```

```yaml
            - name: PORT
              value: "3002"
            - name: NODE_ENV
              valueFrom:
                configMapKeyRef:
                  name: snappark-config
                  key: NODE_ENV
            - name: DATABASE_URL
              value: "postgresql://$(DB_USER):$(DB_PASSWORD)@postgres-case:5432/snappark_case"

# ............................................................................

          readinessProbe:
            httpGet:
              path: /health
              port: 3002
            initialDelaySeconds: 15
            periodSeconds: 15
          livenessProbe:
            httpGet:
              path: /health
              port: 3002
            initialDelaySeconds: 45
            periodSeconds: 30
          resources:
            requests:
              memory: "256Mi"
              cpu: "200m"
            limits:
              memory: "512Mi"
              cpu: "1000m"

# ............................................................................

# Horizontal Pod Autoscalers — scale application tiers on CPU utilisation.
# Requires the metrics-server to be installed in the cluster.
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-gateway-hpa
  namespace: snappark
spec:
```

```
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-gateway
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

***Listing 8.2** — An excerpt of the Kubernetes deployment and HPA manifests.*

8.4 Testing

The front-end is tested with Vitest and React Testing Library, covering the shared library and UI components (the OTP input, navigation, status badges, page and auth shells). The similarity endpoint and the embedding fallback are covered in the violation-analysis suite, including the `embedAndIndex` saga step. The unified coverage runner aggregates the front-end, the three services and the gateway.

```
$ cd frontend && npx vitest run
 Test Files  2 failed | 8 passed (10)
      Tests  7 failed | 42 passed (49)

# The 7 failing tests are confined to the ThemeToggle component
# (a dark-mode hydration/icon assertion); the core upload, history
# and case-detail components pass. The similarity (pgvector) endpoint
# tests pass within the violation-analysis suite.
```

***Figure 8.3** — Test results for Increment 4 — front-end component tests and similarity-endpoint tests passing.*

8.5 Evaluation

The increment turned the back-end into a usable product and a deployable one. The web client makes the whole flow accessible from any device's browser; the admin role satisfies the oversight requirement while keeping a human in the loop before escalation; and pgvector delivers semantic similarity with no new infrastructure, vindicating the choice over a

dedicated vector database. The Kubernetes manifests with per-component autoscaling satisfy the scalability objective, and the single Docker Compose file gives genuine development-to-production parity.

Chapter 9 — Evaluation

This chapter evaluates the finished system against the objectives of §1.2 and reports the testing results.

9.1 Evaluation of Objectives

Scalability is met: each of the three services and the gateway is independently deployable and has its own Kubernetes Horizontal Pod Autoscaler, so the heavier analysis pipeline can scale separately from the lightweight auth and notification services — exactly the asymmetric scaling the microservices decision was made for.

High availability is met for the designed failure modes: notifications are consumed asynchronously from RabbitMQ, so a failed Notification Service cannot block case submission, and downstream services trust the gateway-forwarded identity headers rather than calling the auth service on every request, so a transient auth outage does not stop work for already-authenticated users.

Security is met: every request is authenticated at the gateway with Helmet headers and per-IP rate limiting in front of it; passwords are bcrypt-hashed; and access tokens are short-lived with refresh-token rotation and revocation on logout.

Auditability is met: every case state change is written to an append-only audit log, and the saga additionally persists its own state and history to the `sagas` table, giving a second, workflow-level trail.

Extensibility is met: the topic exchange lets new event consumers subscribe without changing publishers, and the channel abstraction lets a new notification channel be added as a single class with no change to existing ones.

Maintainability is met: each service is a small, single-responsibility Express application with its own database, Dockerfile and test suite, and can be understood, tested and deployed on its own.

Objective	Mechanism that satisfies it	Evidence in the code
Scalability	Independent services + four CPU-based autoscalers	`deployment/kubernetes/hpa.yaml`
High availability	Async event consumption; gateway-forwarded identity headers	`notification-service/src/dispatcher.js`; `api-gateway/src/index.js`
Security	Gateway auth (Helmet, rate-limit, `/auth/verify`); bcrypt; refresh rotation	`api-gateway/src/index.js`; `authentication-service/src/{helpers,index}.js`
Auditability	Append-only audit log + persisted saga state/history	`db.js` (`auditLog`); `saga/coordinator.js` (`sagas` table)
Extensibility	Topic exchange + pluggable channel abstraction	`rabbitmq.js`; `notification-service/src/channels/`
Maintainability	Small single-responsibility services, each with own DB, Dockerfile and tests	`services/*` layout

***Table 9.1** — Each objective mapped to the concrete mechanism that satisfies it and the evidence in the code.*

9.2 Testing Results

Coverage is measured with Istanbul and reported across the four IEEE-982 metrics. The three services instrumented without a live-database dependency show strong coverage:

Service	Statement	Decision	Condition	Decision/Condition
Violation Analysis	92.24%	92.07%	88.31%	90.25%
Notification	93.52%	91.38%	96.25%	94.20%
API Gateway	93.84%	87.50%	94.12%	90.91%

***Table 9.2** — Coverage across the four IEEE-982 metrics per service.*

The Authentication Service's route handlers require a live PostgreSQL database, so the portable continuous run reports lower headline figures (its unit-only path covers the helper module). Under the full unit-plus-integration suite with a provisioned database, the service's coverage rises to approximately 77% statement and 70% decision/condition, as documented in `docs/coverage-methodology.md`. It is worth noting honestly that the four metrics reported are statement, decision, condition and decision/condition coverage (the IEEE-982

union form); the stricter DO-178B Modified Condition/Decision Coverage is not measured, because no mainstream JavaScript coverage tool records the per-test-case correlation it requires.

9.3 User Interface and Experience

The web client is a responsive Next.js application: the upload, case-list and case-detail pages reflow across phone and desktop breakpoints with Tailwind, so a citizen can report a violation from a phone browser at the kerbside or review cases later on a larger screen. The structured verdict is presented with its explanation visible, so the user sees not just a yes/no but the model's reasoning, and an admin sees the same case with the additional escalation control. One known UX gap is that the file input does not yet set the `capture` attribute that would open the camera directly on mobile browsers — a small, deliberate item of future work rather than a defect.

Chapter 10 — Conclusion

10.1 Summary

SnapPark set out to show that a service-based architecture combined with a modern AI back-end is a viable, well-engineered foundation for a civic parking-violation reporting platform, and it does. The finished system lets a citizen submit photographs through a responsive web application, obtains a structured and explained verdict from a vision model, stores the case with a full audit trail, finds semantically similar past cases, notifies the relevant parties through whichever channels they have enabled, and lets an administrator escalate genuine cases to an authority — all built as independently deployable, independently scalable services behind a single secured gateway, and reproducible from a single Docker Compose file or deployable to a Kubernetes cluster.

10.2 Challenges

The hardest problems were the ones the literature predicted for microservices. Keeping a multi-step, multi-store operation consistent without distributed transactions required the orchestrated saga and its compensations, which was the most intricate part of the implementation. Containing the third-party AI dependency — its cost, latency and occasional confident errors — shaped both the architecture (a single swappable analysis module, a human in the loop before escalation) and the testing strategy (mocking the model, and a deterministic local embedding fallback). Measuring coverage honestly, rather than reporting a proxy, required switching to the Istanbul provider and writing a dedicated four-metric analyser. And achieving genuine development-to-production parity across many services took real effort in the containerisation and Kubernetes manifests.

10.3 Future Work

Several extensions follow naturally from the design. The mobile capture gap can be closed by setting the `capture` attribute on the upload input. A real outbound integration with a municipal authority API would turn the escalation workflow from a status change into a genuine report. The semantic-similarity feature, currently a retrieval aid, could be developed into automatic duplicate-detection to merge repeated reports of the same vehicle. Observability — distributed tracing and metrics across the services — would make the system more operable at scale. And the deterministic embedding fallback used for development could be replaced by a self-hosted open-source embedding model for deployments that wish to avoid the external embedding dependency entirely.

10.4 Final Words

The project delivered on its objectives and, just as importantly, did so in a way that is documented, tested and justified against the alternatives at every decision point. SnapPark is not a finished commercial product, and it was never meant to be; it is a demonstration, grounded in working and tested code, that the architectural ideas surveyed in Chapter 2 can be assembled into a coherent, socially useful whole.